

详细设计文档

引言

1.1 编制目的

1.2 词汇表

1.3 参考资料

产品概述

体系结构设计概述

结构视角

4.1 展示层的分解

4.1.1 userui

4.1.1.1 模块概述

4.1.1.2 整体结构

4.1.1.3 模块内部类的接口规范

4.1.2 storeui

4.1.2.1 模块概述

4.1.2.2 整体结构

4.1.2.3 模块内部类的接口规范

4.1.3 productui

4.1.3.1 模块概述

4.1.3.2 整体结构

4.1.3.3 模块内部类的接口规范

4.1.4 orderui

4.1.4.1 模块概述

4.1.4.2 整体结构

4.1.4.3 模块内部类的接口规范

4.1.5 couponui

4.1.5.1 模块概述

4.1.5.2 整体结构

4.1.5.3 模块内部类的接口规范

4.2 业务逻辑层的分解

4.2.1 userbl

4.2.1.1 模块概述

4.2.1.2 整体结构

4.2.1.3 模块内部类的接口规范

4.2.1.4 业务逻辑层的动态模型

4.2.1.5 业务逻辑层的设计原理

4.2.2 storebl

4.2.2.1 模块概述

4.2.2.2 整体结构

4.2.2.3 模块内部类的接口规范

4.2.2.4 业务逻辑层的动态模型

4.2.2.5 业务逻辑层的设计原理

4.2.3 productbl

4.2.3.1 模块概述

4.2.3.2 整体结构

4.2.3.3 模块内部类的接口规范

4.2.3.4 业务逻辑层的动态模型

4.2.3.5 业务逻辑层的设计原理

4.2.4 orderbl

4.2.4.1 模块概述

4.2.4.2 整体结构

4.2.4.3 模块内部类的接口规范

4.2.4.4 业务逻辑层的动态模型

4.2.4.5 业务逻辑层的设计原理

4.2.5 toolsbl

4.2.5.1 模块概述

4.2.5.2 整体结构

4.2.5.3 模块内部类的接口规范

4.2.5.4 业务逻辑层的动态模型

4.2.5.5 业务逻辑层的设计原理

4.2.6 couponbl

4.2.6.1 模块概述

4.2.6.2 整体结构

4.2.6.3 模块内部类的接口规范

4.2.6.4 业务逻辑层的动态模型

4.2.6.5 业务逻辑层的设计原理

4.2.7 apipaybl

4.2.7.1 模块概述

4.2.7.2 整体结构

4.2.7.3 模块内部类的接口规范

4.2.7.4 业务逻辑层的动态模型

4.2.7.5 业务逻辑层的设计原理

4.3 数据层的分解

4.3.1 userdata

4.3.1.1 模块概述

4.3.1.2 整体结构

4.3.1.3 模块内部类的接口规范

4.3.2 storedata

4.3.2.1 模块概述

4.3.2.2 整体结构

4.3.2.3 模块内部类的接口规范

4.3.3 productdata

4.3.3.1 模块概述

4.3.3.2 整体结构

4.3.3.3 模块内部类的接口规范

4.3.4 orderdata

4.3.4.1 模块概述

4.3.4.2 整体结构

4.3.4.3 模块内部类的接口规范

4.3.5 commentdata

4.3.5.1 模块概述

4.3.5.2 整体结构

4.3.5.3 模块内部类的接口规范

4.3.6 coupondata

4.3.6.1 模块概述

4.3.6.2 整体结构

4.3.6.3 模块内部类的接口规范

4.3.7 couponsetdata

4.3.7.1 模块概述

4.3.7.2 整体结构

4.3.7.3 模块内部类的接口规范

依赖视角

引言

1.1 编制目的

本报告详细完成对 **蓝鲸(BlueWhale)商场网购平台系统** 的详细设计，达到指导后续软件构造的目的，同时实现和测试人员及用户的沟通。

本报告面向开发人员、测试人员及最终用户而编写，是了解系统的导航。

1.2 词汇表

详细设计描述的约定详见下表所示：

词汇名称	对应全称或具体含义
BlueWhale	蓝鲸商场网购平台

1.3 参考资料

1. IEEE标准
2. 《软件工程与计算（卷二）软件开发的技术基础》
3. BlueWhale 需求规格说明书
4. BlueWhale 软件体系结构设计文档

产品概述

参考 BlueWhale 网购平台系统的用例文档和软件需求规格说明文档中对产品的概括描述。

体系结构设计概述

参考 BlueWhale 网购平台系统的概要设计文档中对体系结构设计的概述。

结构视角

4.1 展示层的分解

展示层的开发包图参见体系结构设计文档

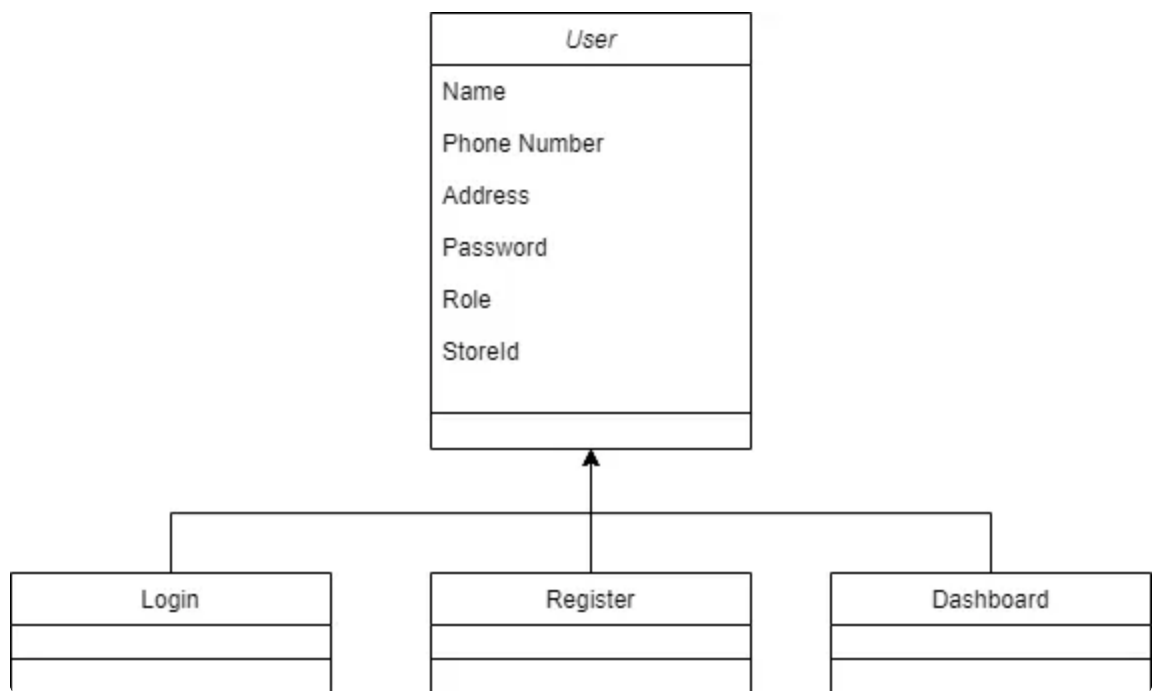
4.1.1 userui

4.1.1.1 模块概述

Userui主要需求：开发和维护系统的用户界面，处理用户在界面上的输入和事件，与 userbl 模块进行数据交互，控制界面的视觉样式

Userui主要职责：提供用户友好的界面交互，与业务逻辑层进行数据交互，管理界面样式和布局

4.1.1.2 整体结构



各个类的职责：

user	获取用户信息
login	登录
register	注册
dashboard	展示用户信息

4.1.1.3 模块内部类的接口规范

接口规范：

接口名称	语法	前置条件	后置条件

user.userRegister	userRegister(RegisterInfo) type RegisterInfo = { role: string, name: string, phone: string, password: string, address: string, storeId?: number, }	输入合法	将注册成功的用户信息传输到后端
user.userLogin	userLogin(loginInfo) type LoginInfo = { phone: string, password: string }	输入合法	验证是否登录成功并获取对应token
user.userInfo	userInfo()	无, 登录	获取当前用户的信息
user.getUserById	getUserById(id)	输入合法, 登录	获取对应用户的部分信息
user.userInfoUpdate	userInfoUpdate(updateInfo)	输入合法, 登录	将更新的用户信息传输给后端

服务接口:

服务名	服务
-----	----

user.userRegister	将注册成功的用户信息传输到后端
user.userLogin	验证是否登录成功并获取对应token
user.userInfo	获取当前用户的信息
user.getUserById	获取对应用户的部分信息

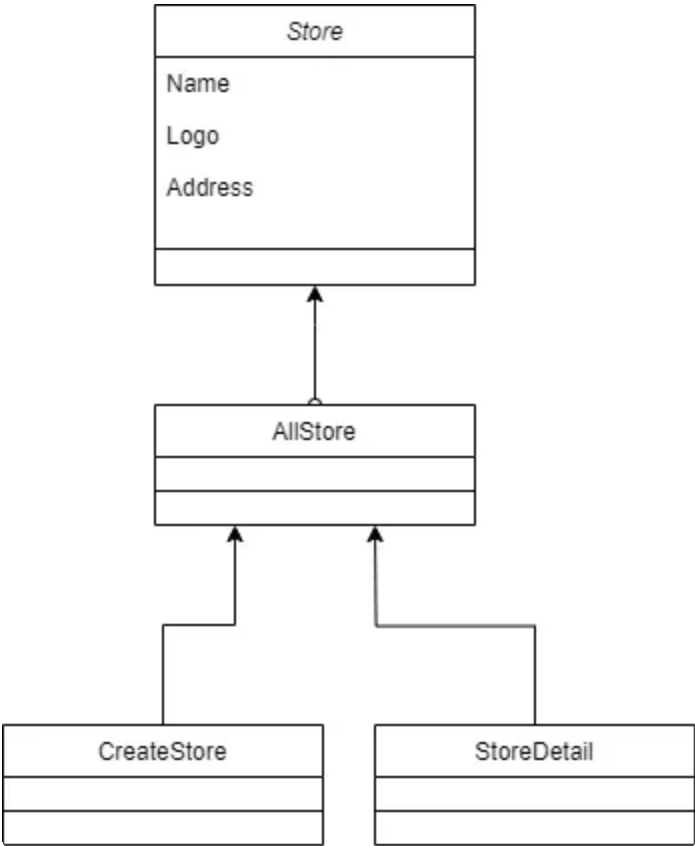
4.1.2 storeui

4.1.2.1 模块概述

Storeui 模块的主要职责: 提供用户友好的商城界面交互, 与 Storesbl 模块进行数据交互, 管理界面样式和布局

Storeui 模块的主要需求: 开发和维护商城系统的用户界面, 处理用户的购物、支付等交易行为, 展示商品、订单、物流等信息

4.1.2.2 整体结构



各个类的职责：

store	含有单个商店信息
AllStore	展示所有商店列表
CreateStore	创建商店
StoreDetail	展示商店具体详情

4.1.2.3 模块内部类的接口规范

接口规范：

接口名称	语法	前置条件	后置条件
------	----	------	------

store.createStore	createStore(storeInfo: StoreInfo) type StoreInfo = { name: string, location: string, logoUrl: string }	输入正确, 登录	创建商店并传输到后端
store.getStoreById	getStoreById(id: number)	输入正确, 登录	获取对应商店
store.getAllStores	getAllStores()	登录	获取商店列表
store.getRatingInfoById	getRatingInfoById(id: number)	输入正确, 登录	获取对应商店评分

服务接口:

服务名	服务
store.createStore	创建商店并传输到后端
store.getStoreById	获取对应商店
store.getAllStores	获取商店列表
store.getRatingInfoById	获取对应商店评分

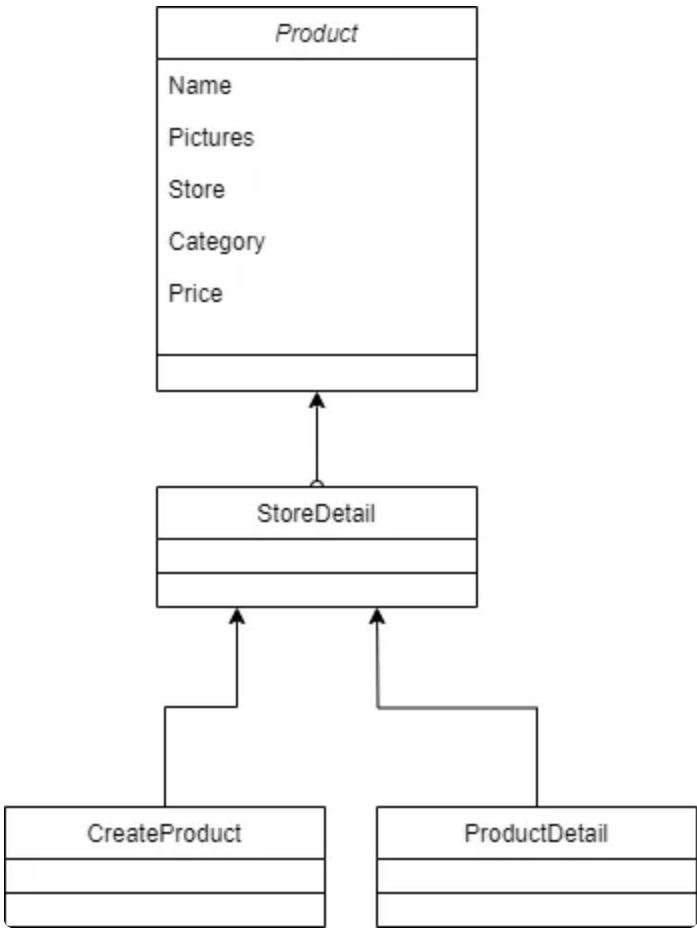
4.1.3 productui

4.1.3.1 模块概述

productui 模块的主要职责：提供产品信息的用户界面交互，与 productbl 模块进行数据交互，管理产品信息的界面样式和布局

productui 模块的主要需求：开发和维护产品信息的用户界面，展示产品的详细信息和属性，处理用户对产品的浏览、搜索、比较等操作

4.1.3.2 整体结构



各个类的职责：

product	含有单个商品信息
StoreDetail	展示商店所有商品列表
CreateProduct	创建商品

ProductDetail	展示商品具体详情
---------------	----------

4.1.3.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
product.createProduct	<pre>createProduct(productInfo : ProductInfo) type ProductInfo = { storeId: number, name: string, category: string, price: number, photoUrlList: string[] }</pre>	输入合法	将添加的商品传输到后端
product.addStock	<pre>addStock(id: number, number: number)</pre>	输入合法	将数据库中对应商品的库存传输到后端
product.getProductByStoreId	<pre>getProductByStoreId(storeId: number)</pre>	输入合法	获取相应商店的商品列表
product.getRatingInfoById	<pre>getRatingInfoById(id: number)</pre>	输入合法	获取相应的评分
product.getProductById	<pre>getProductById(productId: number)</pre>	输入合法	获取相应商品信息

product.getCommentsByProductId	getCommentsByProductId (productId: number)	输入合法	获取相应评论列表信息
--------------------------------	--	------	------------

服务接口:

服务名	服务
product.createProduct	将添加的商品传输到后端
product.addStock	将数据库中对应商品的库存传输到后端
product.getProductByStoreId	获取相应商店的商品列表
product.getRatingInfoById	获取相应的评分
product.getProductById	获取相应商品信息
product.getCommentsByProductId	获取相应评论列表信息

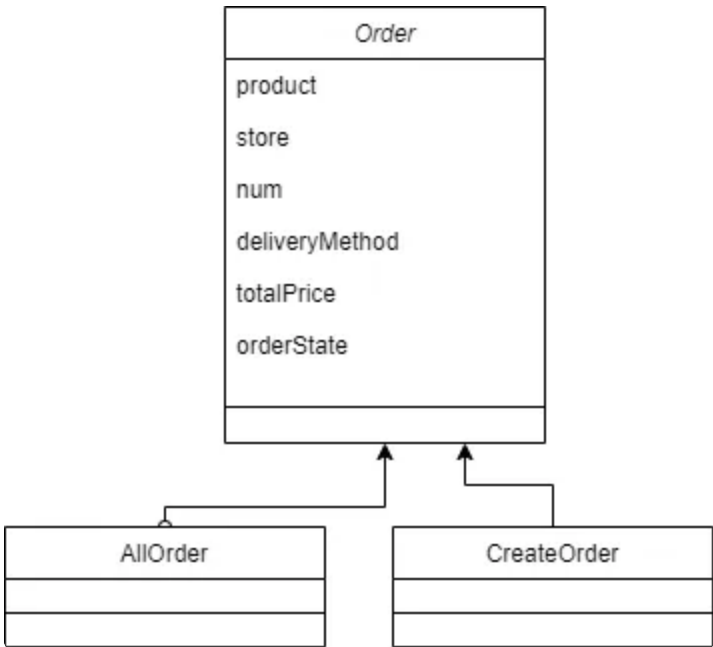
4.1.4 orderui

4.1.4.1 模块概述

orderui 模块的主要职责：提供订单管理的用户界面交互，与 orderservice 模块进行数据交互，管理订单信息的界面样式和布局

orderui 模块的主要需求：开发和维护订单管理的用户界面，展示订单的详细信息和状态，处理用户对订单的查询、修改、取消等操作

4.1.4.2 整体结构



各个类的职责：

order	含有单个订单信息
AllOrder	展示所有订单列表
CreateOrder	创建订单

4.1.4.3 模块内部类的接口规范

接口规范：

接口名称	语法	前置条件	后置条件
------	----	------	------

order.createOrder	<pre>createOrder(orderInfo: OrderInfo) type OrderInfo = { productId: number, storeId: number, num: number, deliveryMethod: string, orderState: string, totalPrice: number }</pre>	输入合法, 用户登录	将添加的订单传输到后端
order.getOrderById	getOrderById(id: number)	输入合法, 登录	获取相应id的订单
order.payOrder	payOrder(orderId: number, couponList: number[], isDirectPay: boolean)	输入合法, 用户登录	将订单更新后的状态传输到后端
order.getAllOrder	getAllOrder()	登录	获取与用户身份相应的订单
order.deliveryOrder	deliveryOrder(id: number)	输入合法, 员工登录	将订单更新后的状态传输到后端

order.receiveOrder	receiveOrder(id: number)	输入合法, 顾客登录	将订单更新后的状态传输到后端
order.commentOnOrder	commentOnOrder(id: number, content: OrderCommentInfo) type OrderCommentInfo = { text: string, rating: number }	输入合法, 顾客登陆	将添加的评论传输到后端

服务接口:

服务名	服务
order.createOrder	将添加的订单传输到后端
order.getOrderById	获取相应id的订单
order.payOrder	将订单更新后的状态传输到后端
order.getAllOrder	获取与用户身份相应的订单
order.deliveryOrder	将订单更新后的状态传输到后端

order.receiveOrder	将订单更新后的状态传输到后端
order.commentOnOrder	将添加的评论传输到后端

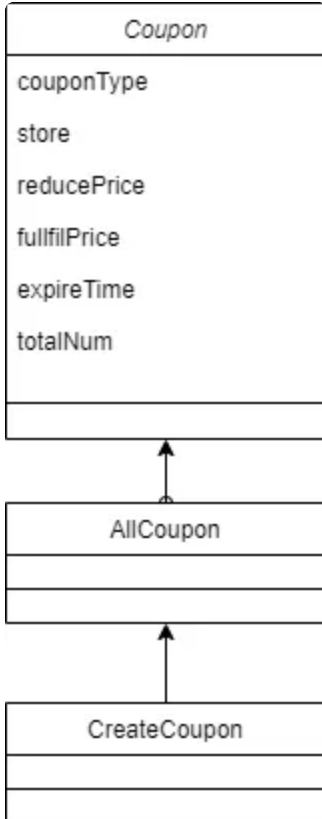
4.1.5 couponui

4.1.5.1 模块概述

couponui模块职责：提供优惠券管理的用户界面交互，与 couponservice 模块进行数据交互，管理优惠券信息的界面样式和布局

couponui模块需求：开发和维护优惠券管理的用户界面，展示可用优惠券的详细信息，处理用户的优惠券领取、使用等操作，与 couponbl 模块交互获取优惠券数据

4.1.5.2 整体结构



各个类的职责：

coupon	含有单个优惠券信息
AllCoupon	展示所有优惠券列表
CreateCoupon	创建优惠券
StoreCoupon	展示优惠券具体详情

4.1.5.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
Coupon.createCoupon	<pre>createCoupon(couponInfo: CouponInfo) type CouponInfo = { couponType: string, reducePrice: number, fulfillPrice: number, expireTime: string, totalNum: number }</pre>	登录为员工或者经理, 输入合法	创建优惠券组
Coupon.getAllCouponSet	getAllCouponSet()	输入合法	获取所有优惠券组
Coupon.getAllCoupon	getAllCoupon(type: string)	顾客登录, 输入合法	获取顾客优惠券列表

Coupon.check	check(setId: number)	输入合法, 登录为顾客	检查优惠券组是否已经被顾客领取
Coupon.getCouponSetById	getCouponSetById(setId: number)	输入合法, 登录	获取优惠券组的信息
Coupon.acquire(Integer setId)	acquire(setId: number)	顾客登录, 输入合法	顾客领取优惠券
Coupon.couponIsValid(Integer setId)	couponIsValid(setId: number)	输入合法	检查优惠券组是否过期

服务接口:

服务名	服务
Coupon.createCoupon	创建优惠券组
Coupon.getAllCouponSet	获取所有优惠券组
Coupon.getAllCoupon	获取顾客优惠券列表
Coupon.check	检查优惠券组是否已经被顾客领取
Coupon.getCouponSetById	获取优惠券组的信息
Coupon.acquire(Integer setId)	顾客领取优惠券
Coupon.couponIsValid(Integer setId)	检查优惠券组是否过期

4.2 业务逻辑层的分解

业务逻辑层的开发包图

4.2.1 userbl

4.2.1.1 模块概述

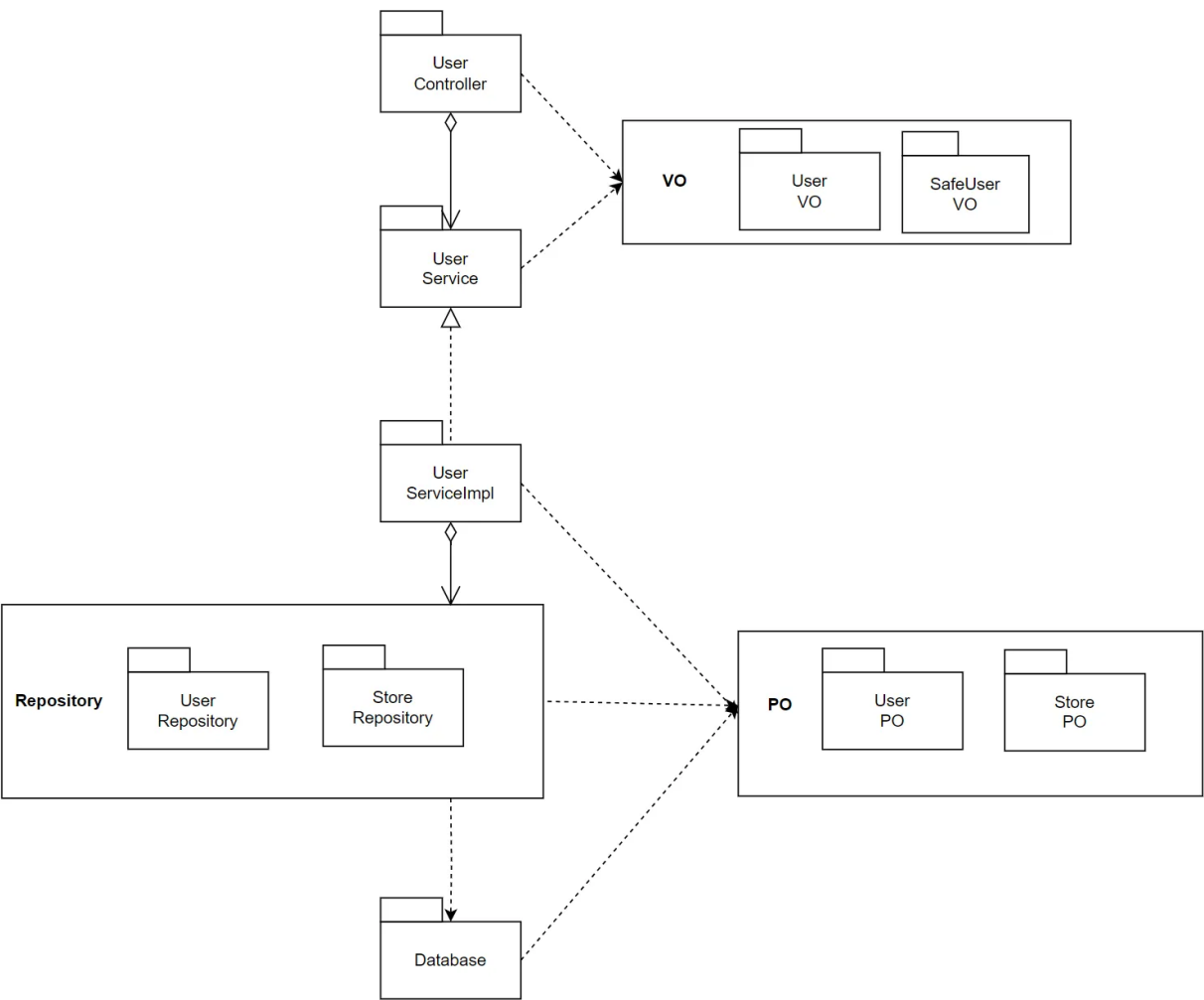
userbl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

userbl 模块的职责及接口参见软件系统结构描述文档。

4.2.1.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口。UserPO 是作为账户信息的持久化对象而添加到设计模型中的。

userbl的模块设计如下：



各个类的职责如下:

UserController	负责实现用户信息前后端的数据交互
UserService	定义用户相关的接口
UserServiceImpl	具体实现用户相关功能
UserPO	用户相关信息的实现载体
UserRepository	实现对于数据库的增删改查
UserVO	用户信息持久化
StorePO	商店相关信息的实现载体
StoreRepository	实现对于数据库的增删改查
StoreVO	商店信息持久化

4.2.1.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
UserController.register	public ResultVO<Boolean> register(@RequestBody UserVO userVO)	输入合法	将注册的用户存入数据库
UserController.login	public ResultVO<String> login(@RequestParam("phone") String phone, @RequestParam("password") String password)	输入合法	验证是否登录成功并获取对应token
UserController.getInformation	public ResultVO<UserVO> getInformation()	无, 登录	获取当前用户的信息
UserController.getUser	public ResultVO<SafeUserVO> getUser(@PathVariable(value = "id") String id)	输入合法, 登录	获取对应用户的部分信息

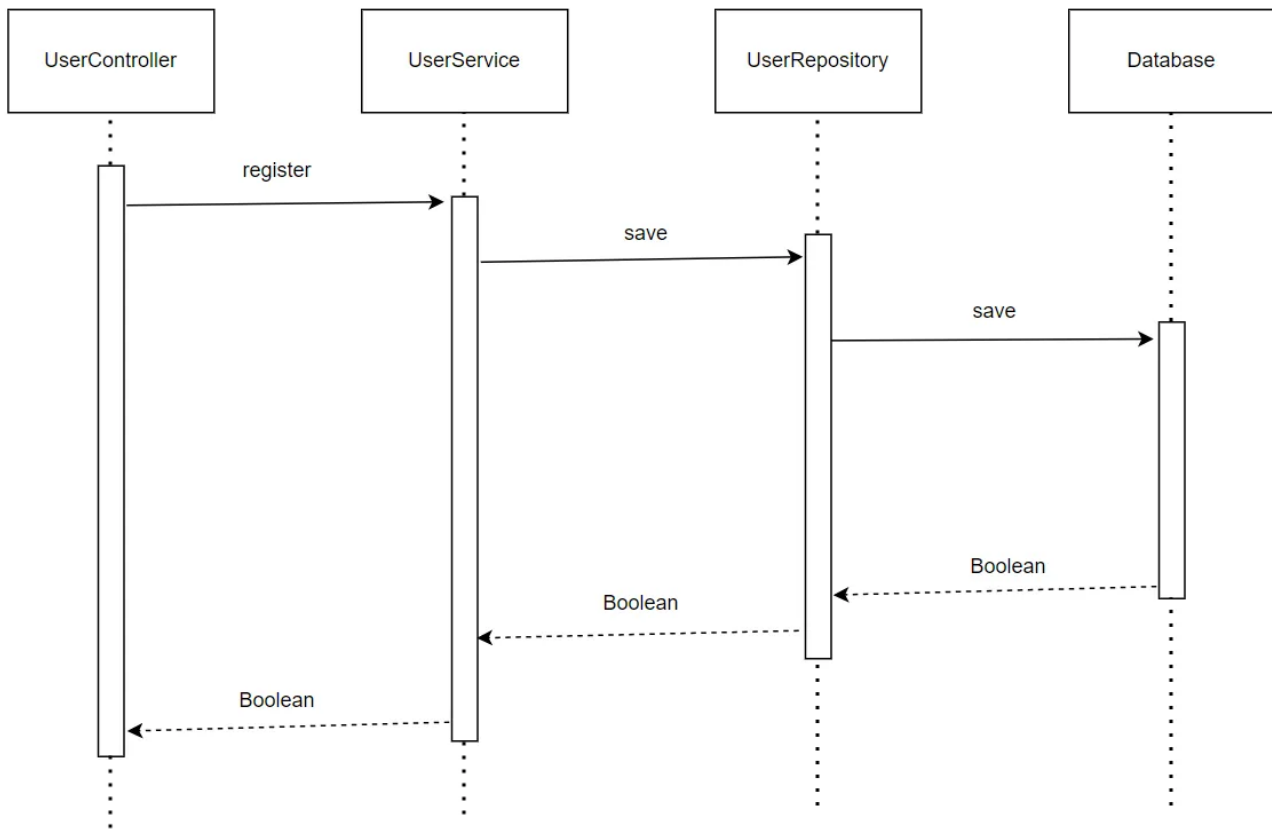
UserController.updtae Information	public ResultVO<Boolean> updateInformation(@RequestBody UserVO userVO)	输入合 法, 登录	将更新的用户信息存入 数据库
--------------------------------------	--	--------------	-------------------

服务接口:

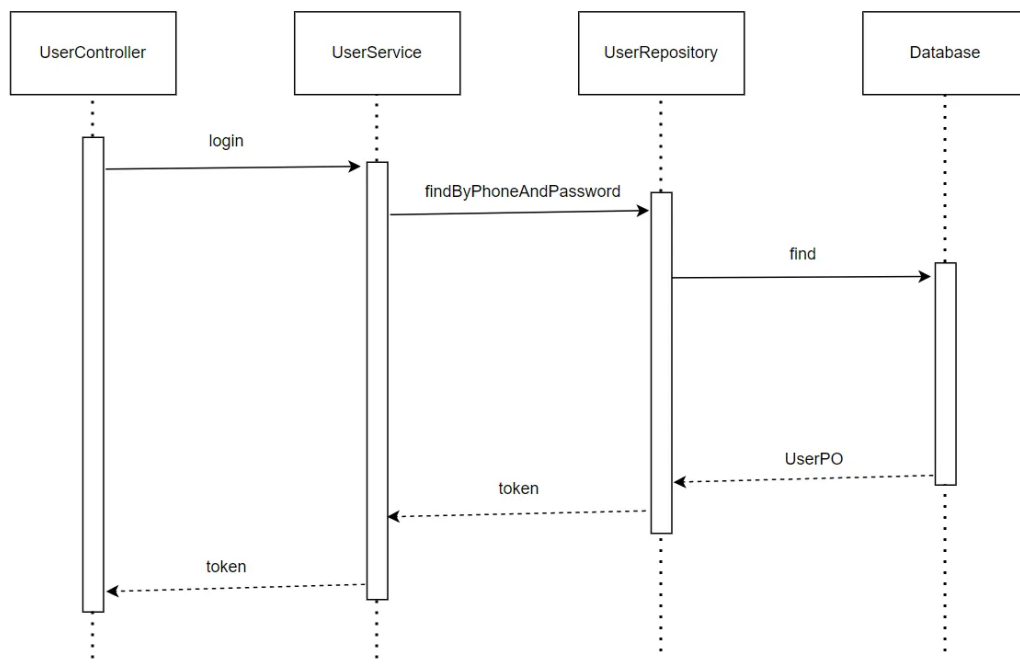
服务名	服务
UserController.register(UserVO userVO)	将注册的用户存入数据库
UserController.login(String phone, String password)	验证是否登录成功并获取对应token
UserController.getInformation()	获取当前用户的信息
UserController.getUser(String id)	获取对应用户的部分信息
UserController.updtaeInformation(UserVO userVO)	将更新的用户信息存入数据库

4.2.1.4 业务逻辑层的动态模型

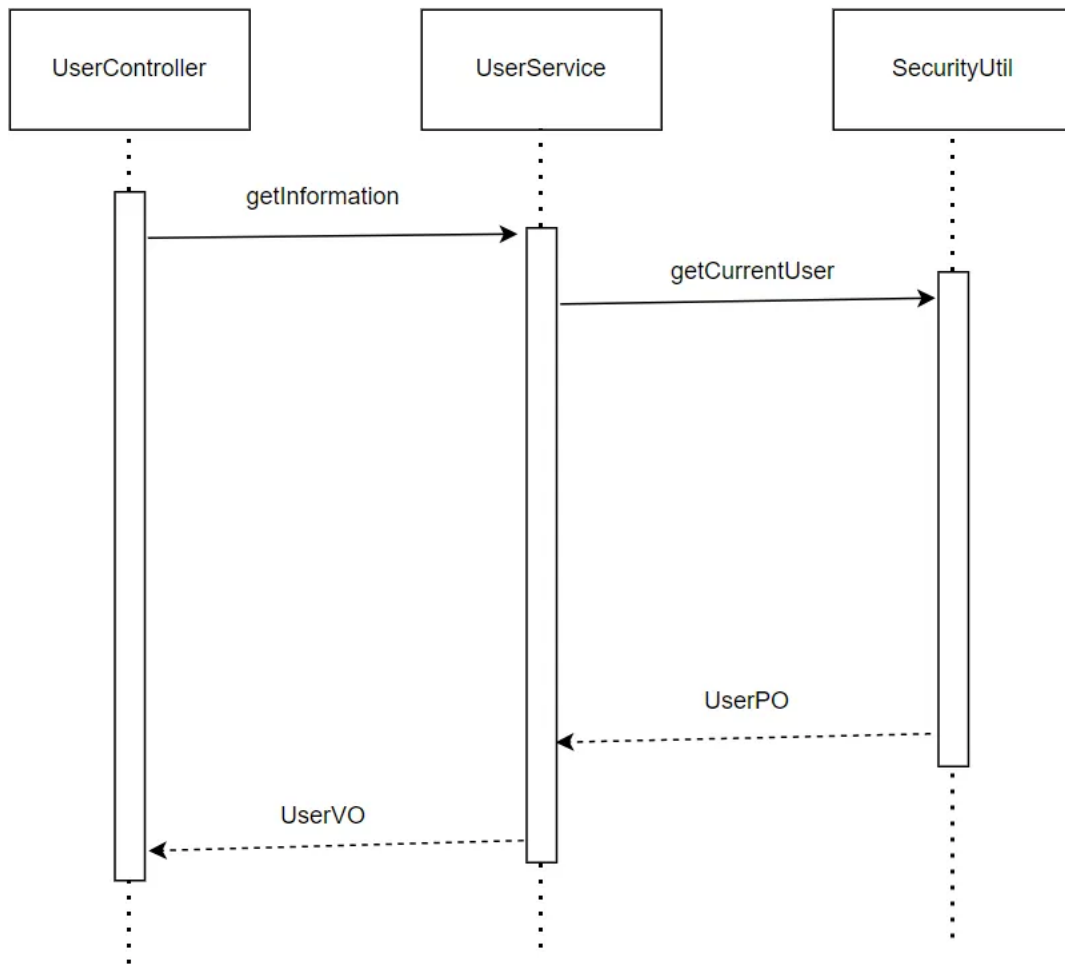
下图表明了 在 BlueWhale 系统中, UserController.register 的相关对象之间的协作。



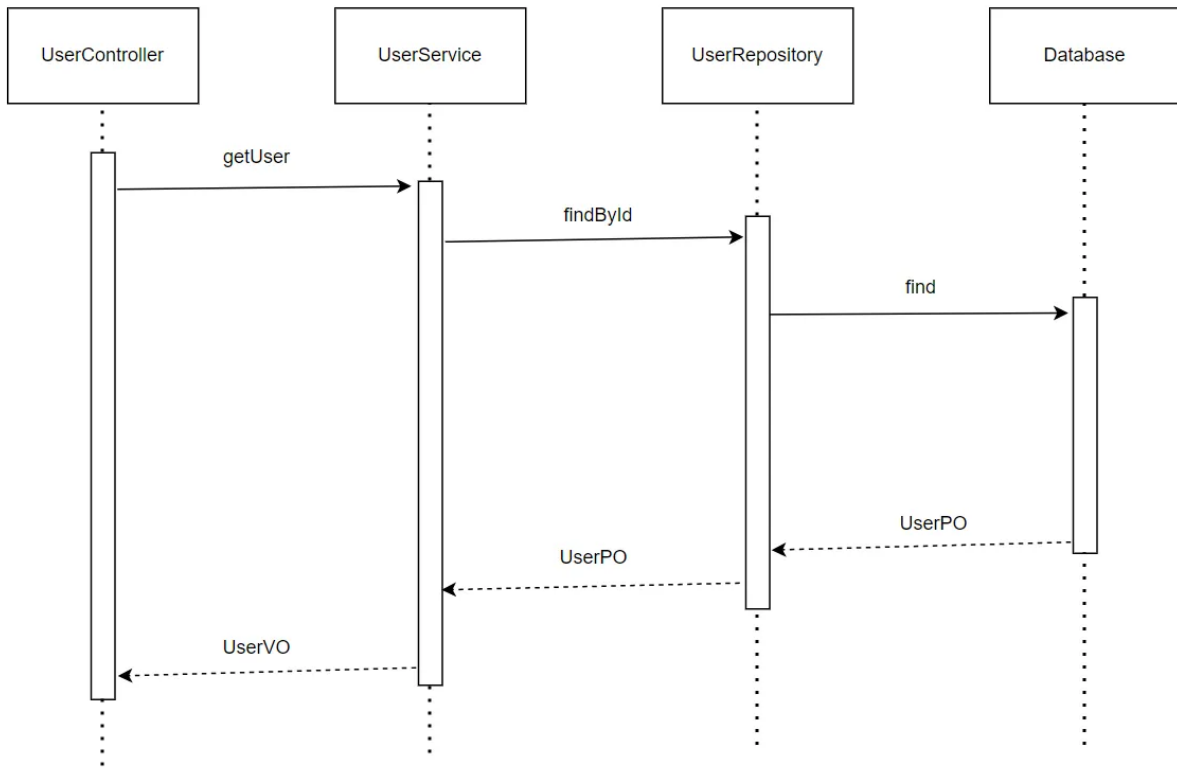
下图表明了 在 ERP 系统中，UserController.login 的相关对象之间的协作。



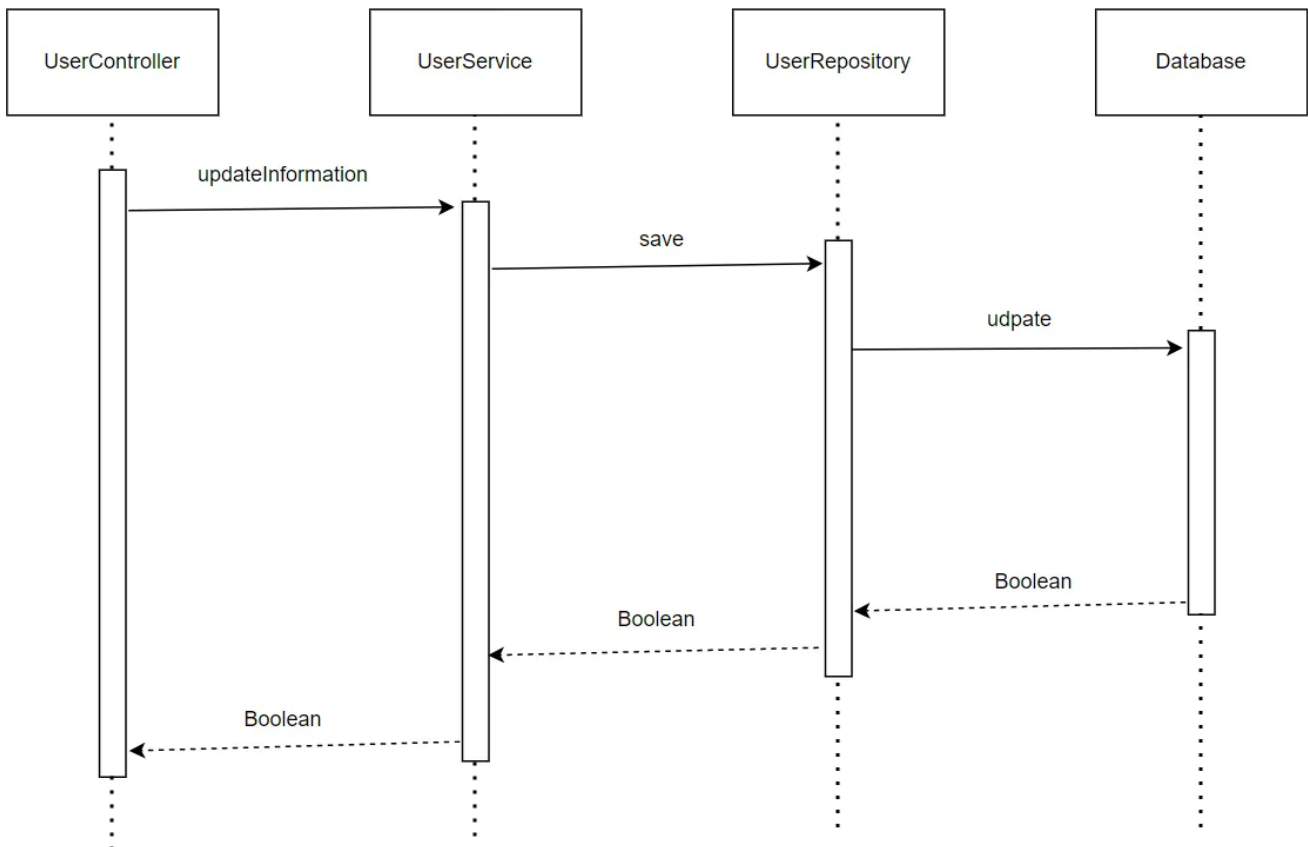
下图表明了 在 BlueWhale 系统中，UserController.getInformation 的相关对象之间的协作。



下图表明了 在 ERP 系统中，UserController.getUser 的相关对象之间的协作。



下图表明了 在 ERP 系统中，UserController.updateInformation的相关对象之间的协作。



4.2.1.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.2.2 storebl

4.2.2.1 模块概述

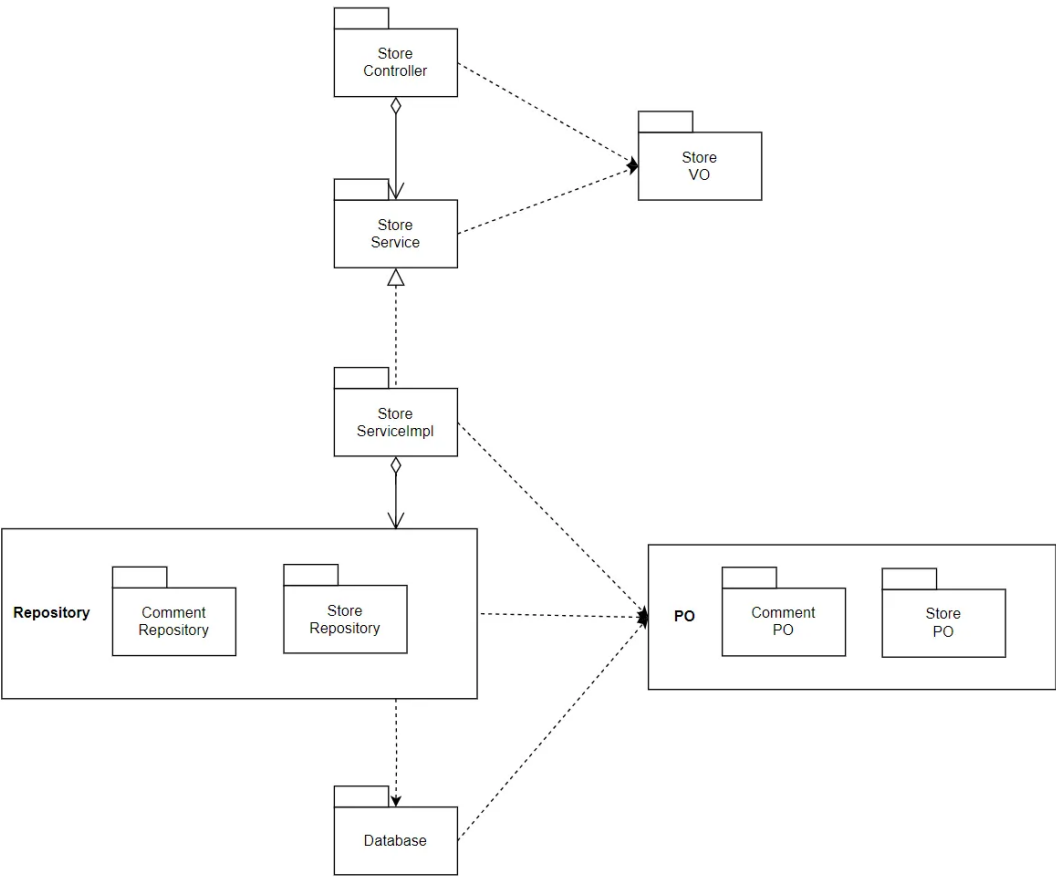
storebl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

storebl 模块的职责及接口参见软件系统结构描述文档。

4.2.2.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口。StorePO 是作为账户信息的持久化对象而添加到设计模型中的。

userbl的模块设计如下:



各个类的职责如下:

StoreController	负责实现商店信息前后端的数据交互
StoreService	定义商店相关的接口
StoreServiceImpl	具体实现商店相关功能
StorePO	商店相关信息的实现载体
StoreRepository	实现对于数据库的增删改查
StoreVO	商店信息持久化
CommentPO	评论相关信息的实现载体
CommentRepository	实现对于数据库的增删改查
CommentVO	评论信息持久化

4.2.2.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
StoreController.create	public ResultVO<Boolean> create(@RequestBody StoreVO storeVO)	输入正 确, 登录	创建商店并存入数据库
StoreController.getStore	public ResultVO<StoreVO> getStore(@PathVariable(value = "id") Integer id)	输入正 确, 登录	获取对应商店
StoreController.getAllStores	public ResultVO<List<StoreVO>> getAllStores()	登录	获取商店列表
StoreController.getRating	public ResultVO<RatingVO> getRating(@PathVariable(value = "id") Integer id)	输入正 确, 登录	获取对应商店评分

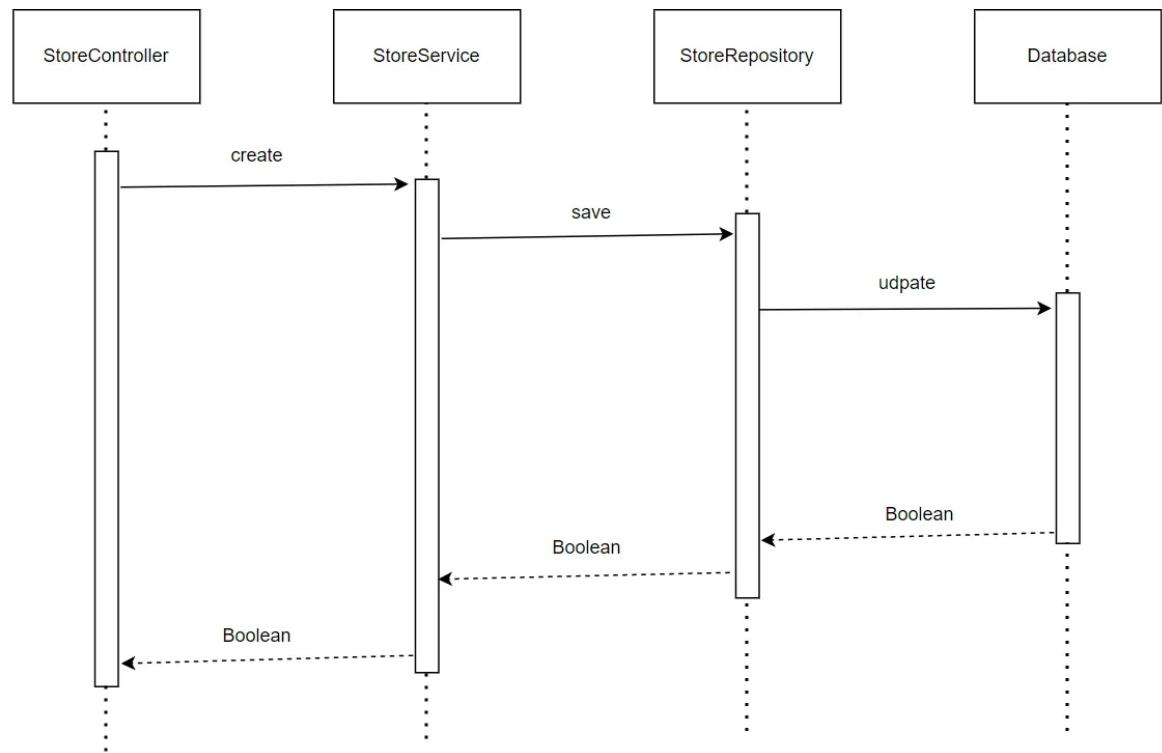
服务接口:

服务名	服务
-----	----

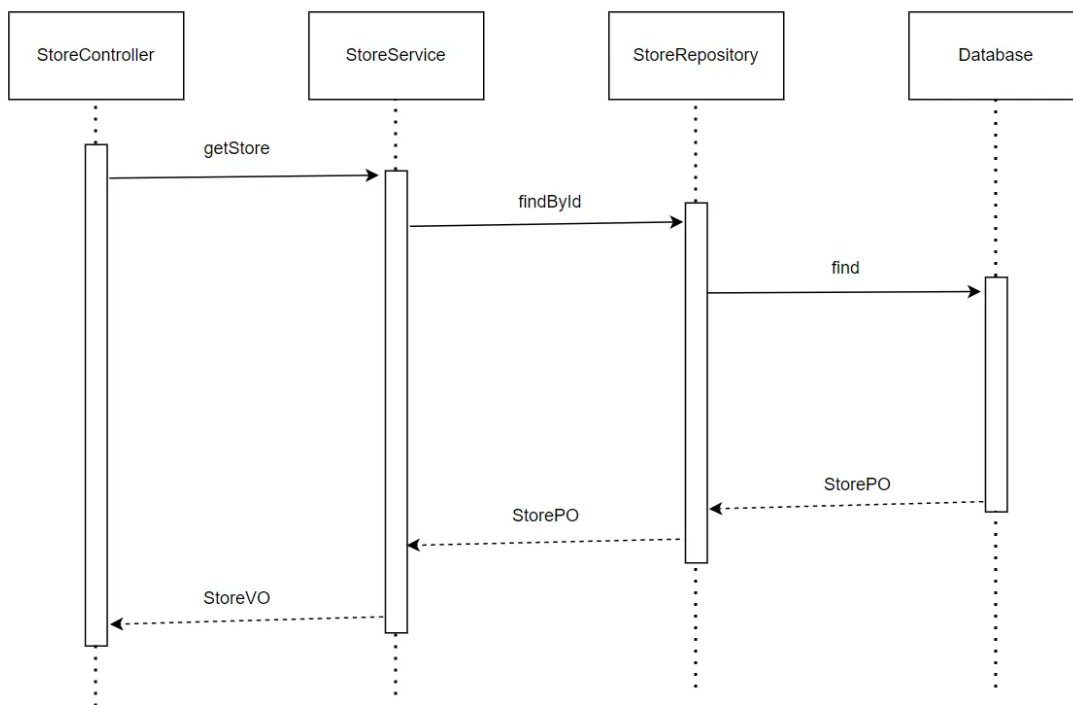
StoreController.create(StoreVO storeVO)	创建商店并存入数据库
StoreController.getStore(Integer id)	获取对应商店
StoreController.getAllStores()	获取商店列表
StoreController.getRating(Integer id)	获取对应商店评分

4.2.2.4 业务逻辑层的动态模型

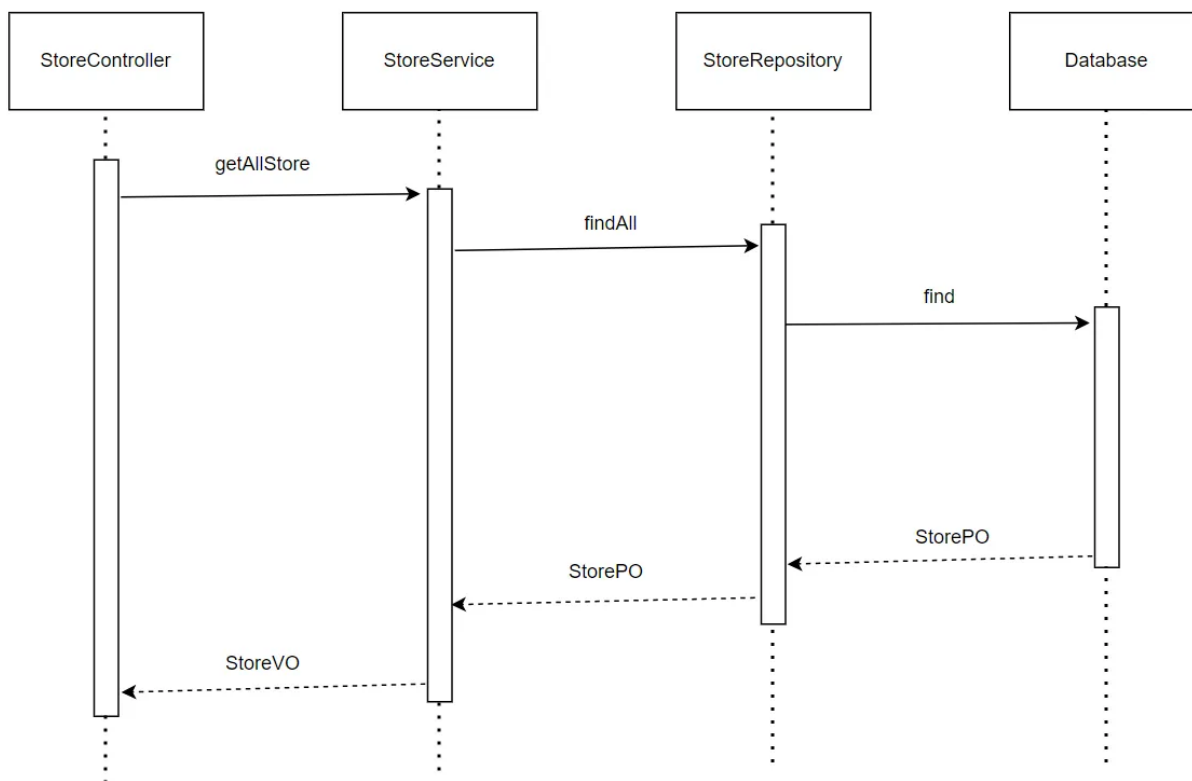
下图表明了 在 BlueWhale 系统中，StoreController.create的相关对象之间的协作。



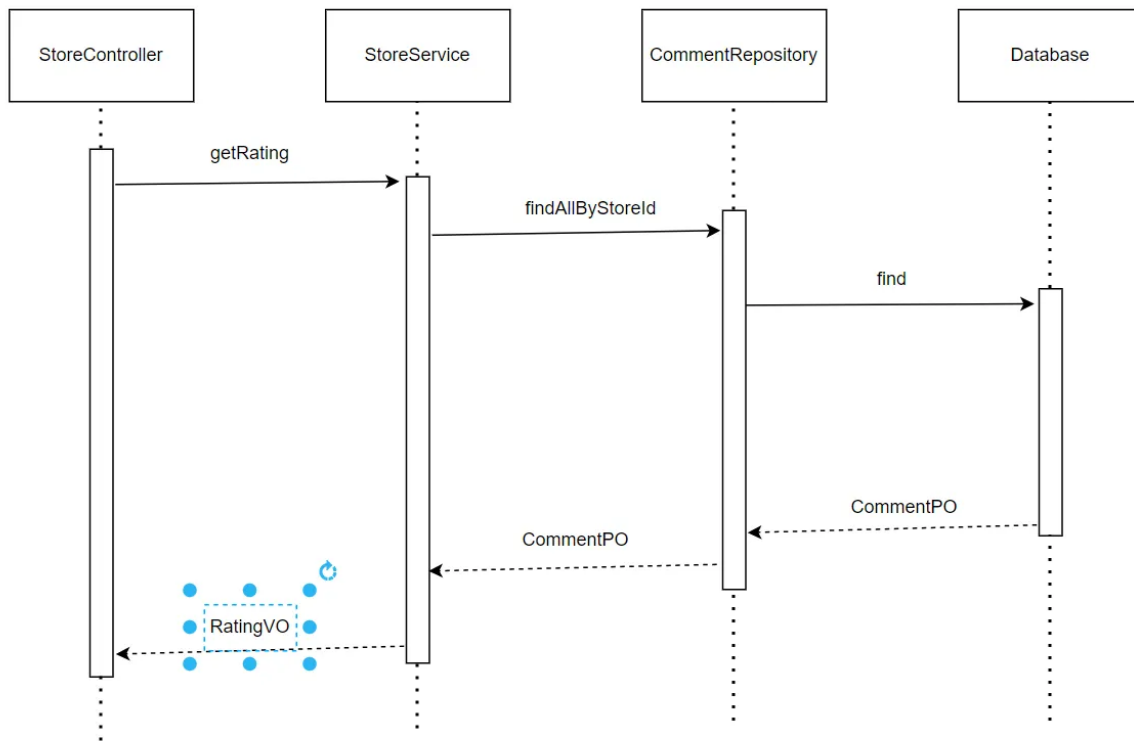
下图表明了 在 BlueWhale 系统中，StoreController.getStore的相关对象之间的协作。



下图表明了 在 BlueWhale 系统中，`StoreController.getAllStores` 的相关对象之间的协作。



下图表明了 在 ERP 系统中，`StoreController.getRating` 的相关对象之间的协作。



4.2.2.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.2.3 productbl

4.2.3.1 模块概述

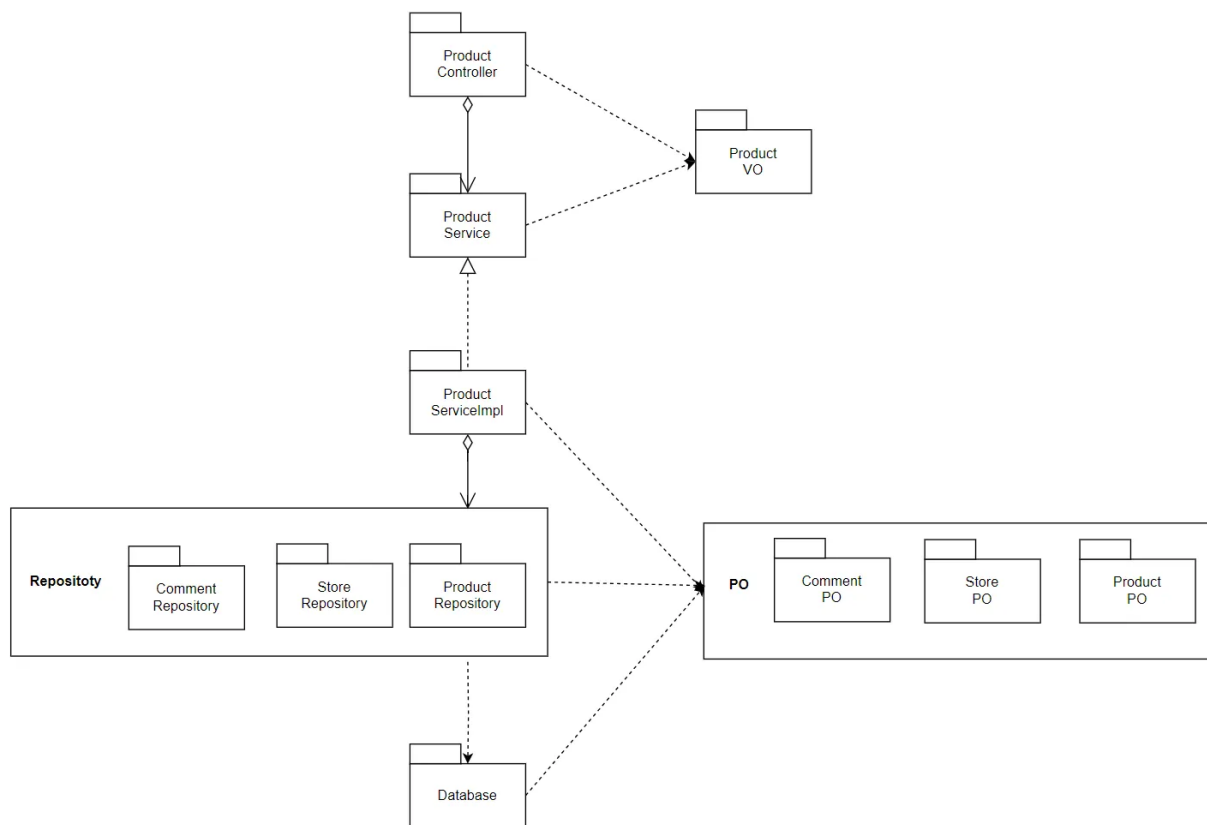
productbl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

productbl 模块的职责及接口参见软件系统结构描述文档。

4.2.3.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口。ProductPO 是作为账户信息的持久化对象而添加到设计模型中的。

userbl的模块设计如下:



各个类的职责如下:

ProductController	负责实现商品信息前后端的数据交互
ProductService	定义商品相关的接口
ProductServiceImpl	具体实现商品相关功能
ProductPO	商品相关信息的实现载体
ProductRepository	实现对于数据库的增删改查
ProductVO	商品信息持久化
StorePO	商店相关信息的实现载体
StoreRepository	实现对于数据库的增删改查
StoreVO	商店信息持久化
CommentPO	评论相关信息的实现载体

CommentReporitor y	实现对于数据库的增删改查
CommentVO	评论信息持久化

4.2.3.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
ProductControlle r.create	public ResultVO<Boolean> create(@RequestBody ProductVO productVO)	输入合法	将添加的商品传入数据库中
ProductControlle r.addStock	public ResultVO<Boolean> addStock(@PathVariable(v alue = "id") Integer id, @RequestParam("number ") Integer number)	输入合法	将数据库中对应该商品的库存更新并 保存
ProductControlle r.getAllProducts	public ResultVO<List<ProductVO >> getAllProducts(@Request Param("storeId") Integer storeId)	输入合法	获取相应的商品列表
ProductControlle r.getRating	public ResultVO<RatingVO> getRating(@PathVariable(value = "id") Integer id)	输入合法	获取相应的评分
ProductControlle r.getProduct	public ResultVO<ProductVO> getProduct(@PathVariable (value = "id") Integer id)	输入合法	获取相应商品信息

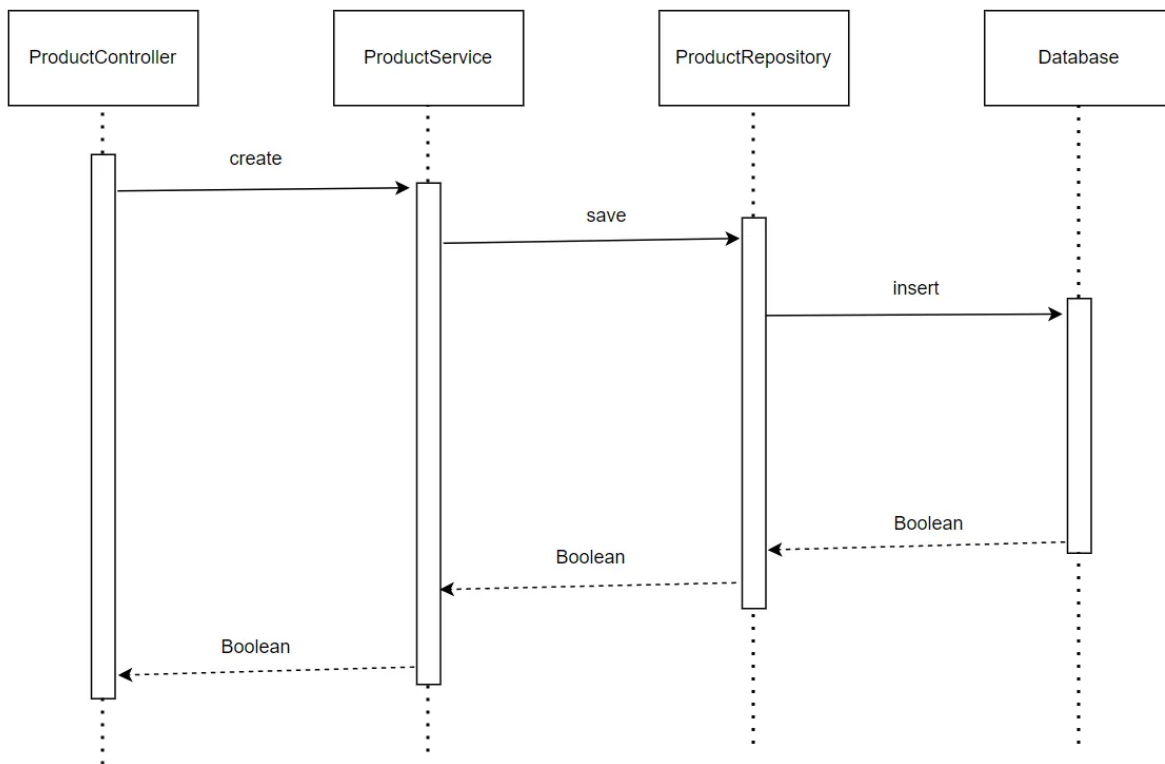
ProductController.getComments	public ResultVO<List<Comment >> getComments(@PathVariable(value = "id") Integer id)	输入合法	获取相应评论列表信息
-------------------------------	---	------	------------

服务接口:

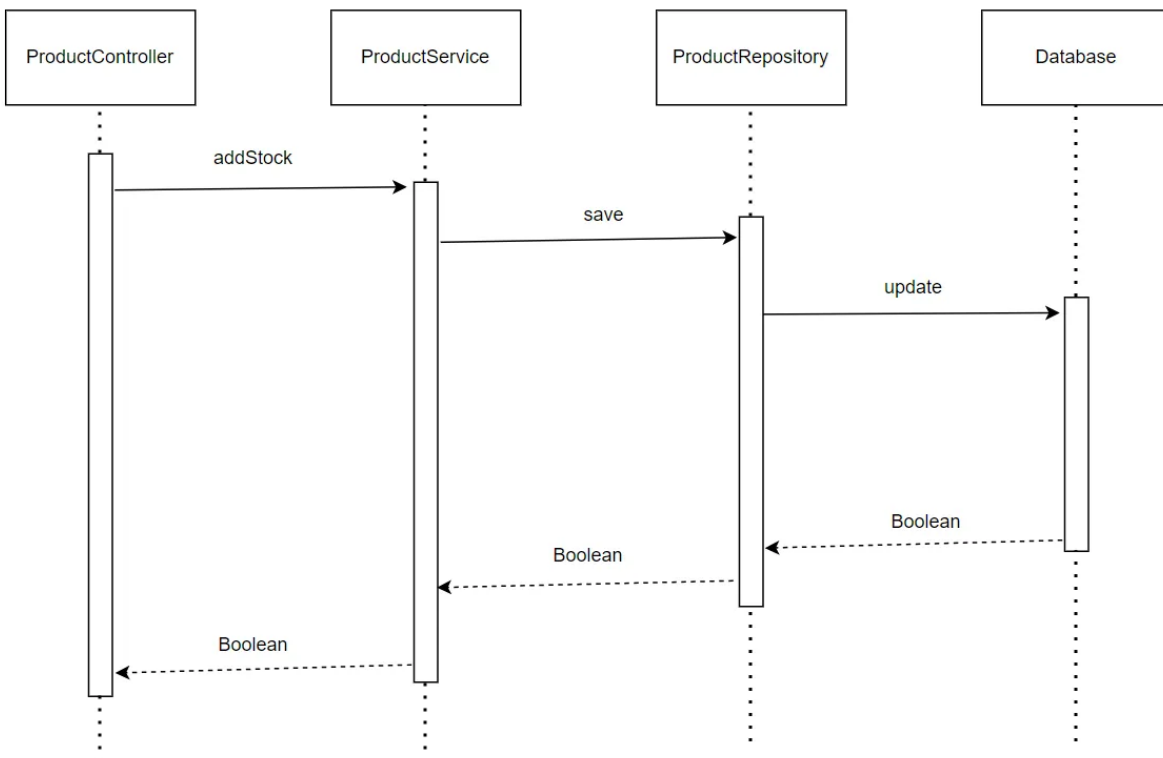
服务名	服务
ProductController.create(ProductVO productVO)	将添加的商品传入数据库中
ProductController.addStock(Integer id, Integer number)	将数据库中对应该商品的库存更新并保存
ProductController.getAllProducts(Integer storeId)	获取相应的商品列表
ProductController.getRating(Integer id)	获取相应的评分
ProductController.getProduct(Integer id)	获取相应商品信息
ProductController.getComments(Integer id)	获取相应评论列表信息

4.2.3.4 业务逻辑层的动态模型

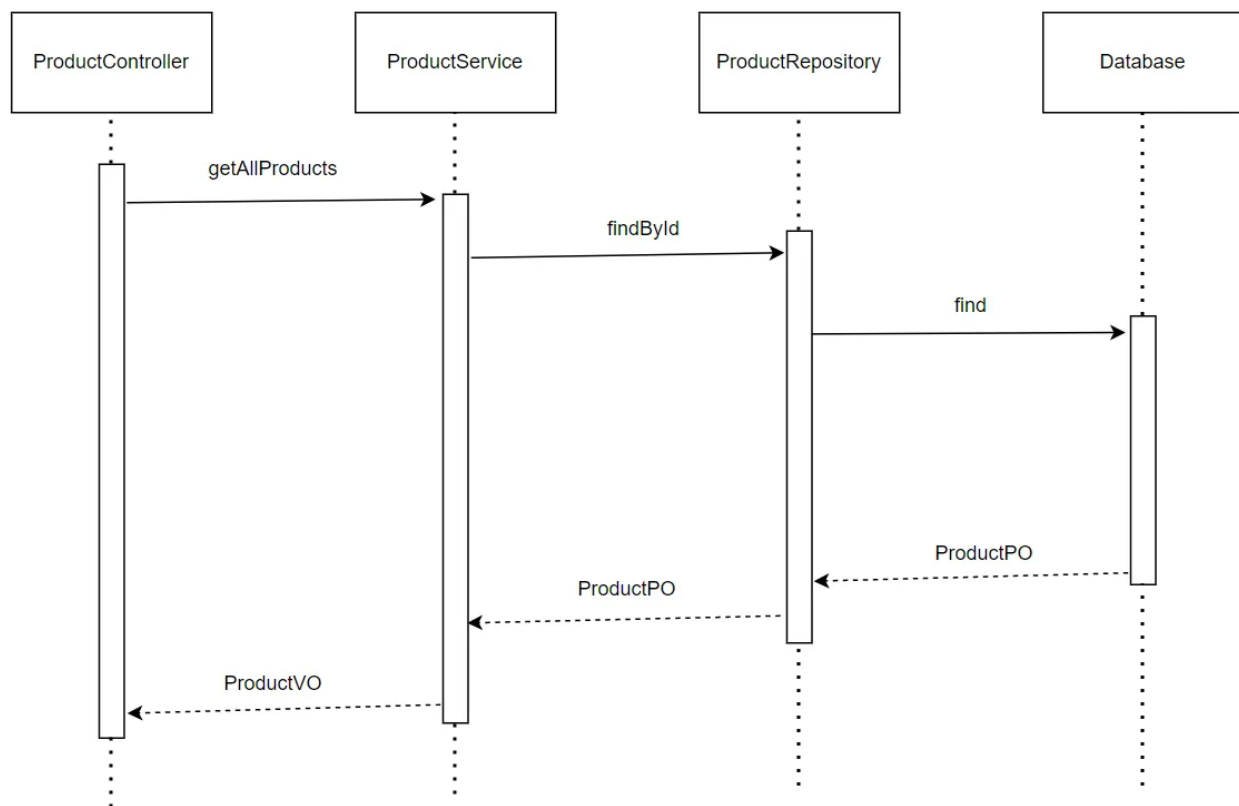
下图表明了 在 BlueWhale 系统中, ProductController.create 的相关对象之间的协作。



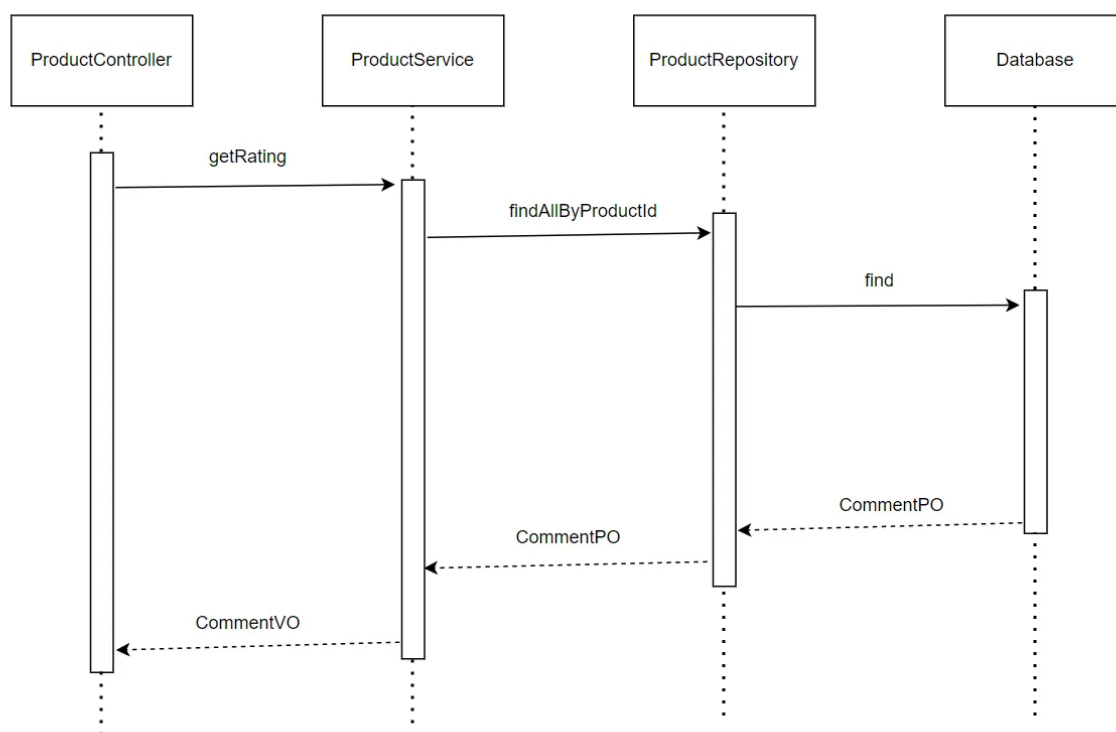
下图表明了 在 BlueWhale 系统中，`ProductController.addStock` 的相关对象之间的协作。



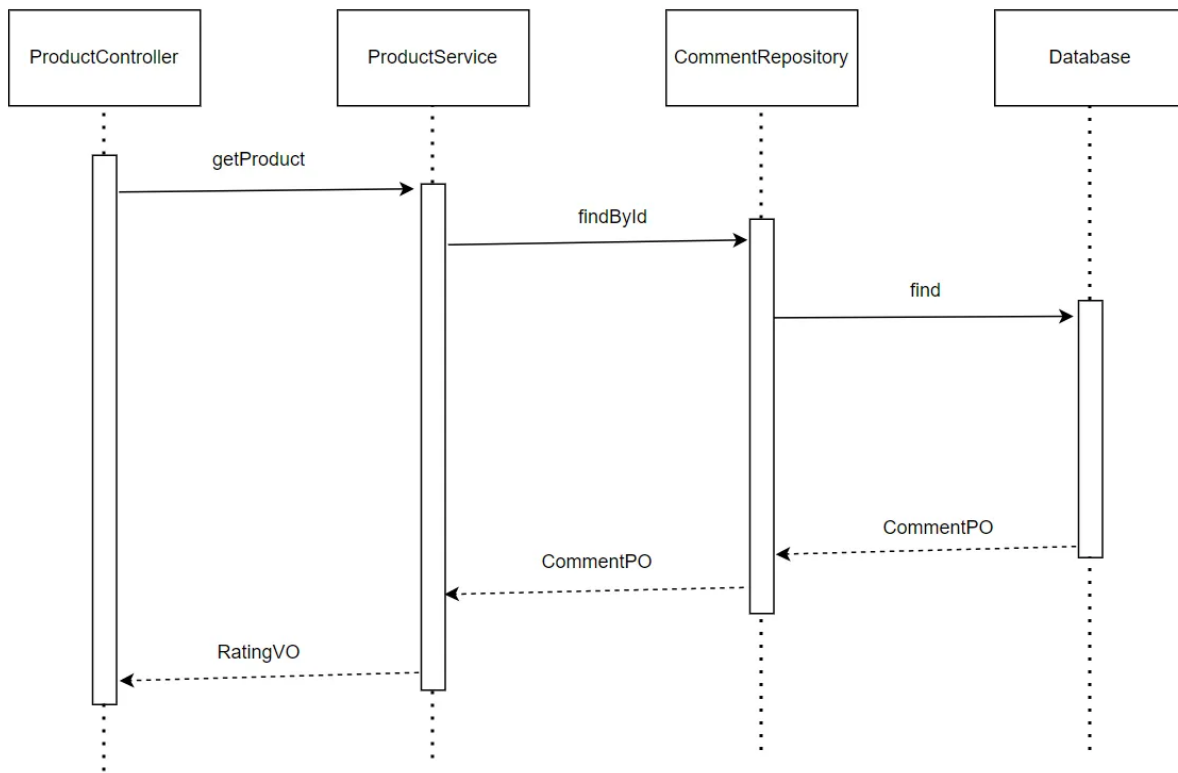
下图表明了 在 ERP 系统中，`ProductController.getAllProducts` 的相关对象之间的协作。



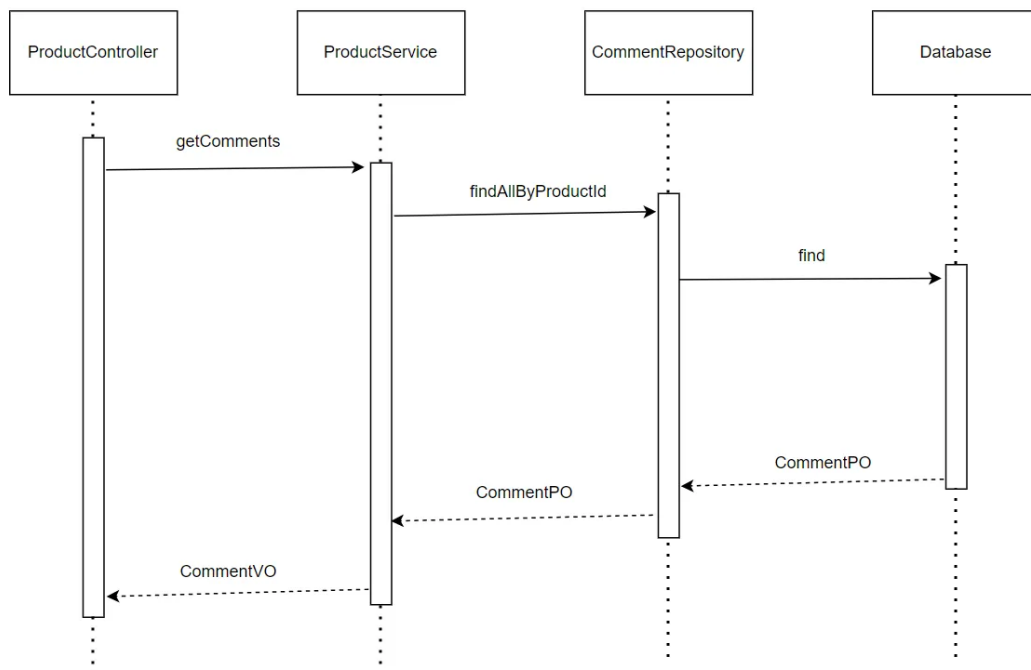
下图表明了 在 BlueWhale 系统中，ProductController.getRating 的相关对象之间的协作。



下图表明了 在 BlueWhale 系统中，ProductController.getProduct 的相关对象之间的协作。



下图表明了 在 ERP 系统中，ProductController.getComments 的相关对象之间的协作。



4.2.3.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.2.4 orderbl

4.2.4.1 模块概述

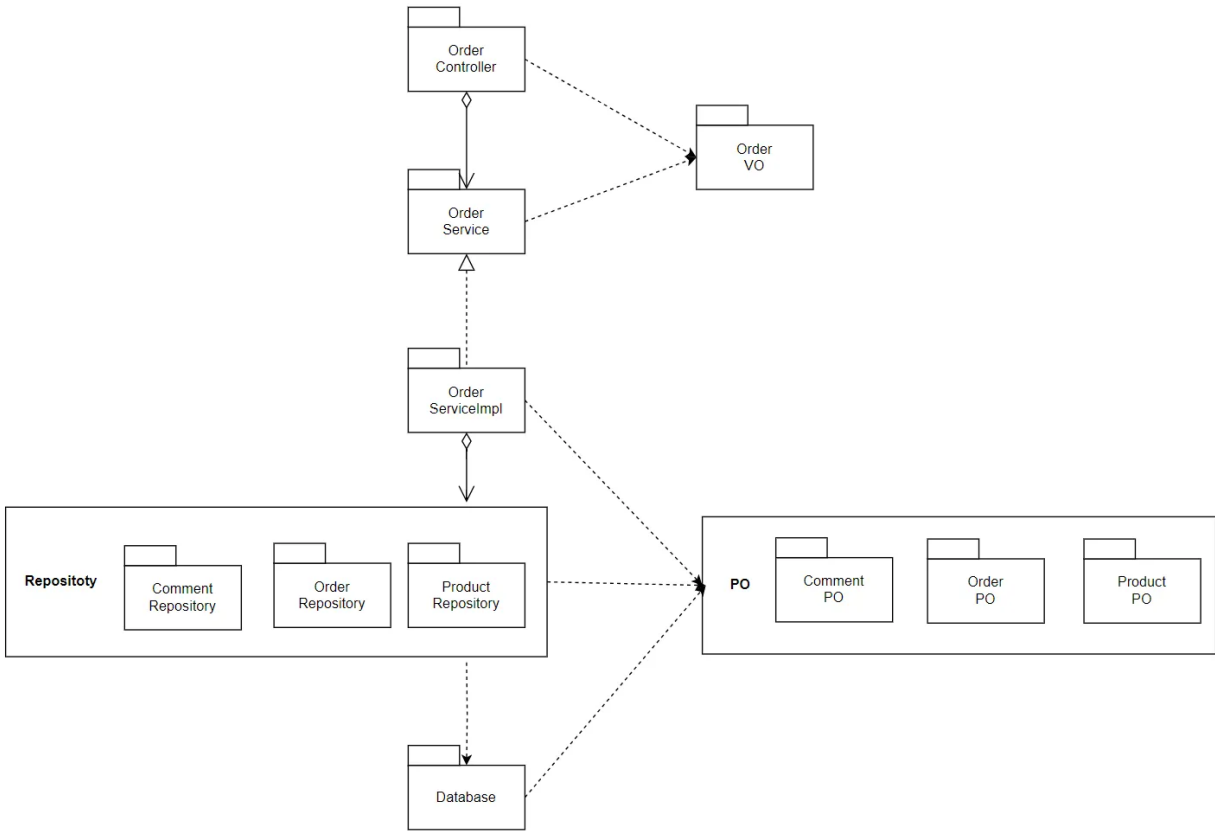
orderbl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

orderbl 模块的职责及接口参见软件系统结构描述文档。

4.2.4.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口。OrderPO 是作为账户信息的持久化对象而添加到设计模型中的。

userbl的模块设计如下：



各个类的职责如下：

OrderController	负责实现订单信息前后端的数据交互
-----------------	------------------

OrderService	定义订单相关的接口
OrderServiceImpl	具体实现订单相关功能
OrderPO	订单相关信息的实现载体
OrderRepository	实现对于数据库的增删改查
OrderVO	订单信息持久化
ProductPO	商品相关信息的实现载体
ProductRepository	实现对于数据库的增删改查
ProductVO	商品信息持久化
CommentPO	评论相关信息的实现载体
CommentRepository	实现对于数据库的增删改查
CommentVO	评论信息持久化

4.2.4.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
OrderController.create	<code>public ResultVO<Integer> create(@RequestBody OrderVO order)</code>	输入合法, 用户登录	将添加的订单传给数据库
OrderController.getOrderById	<code>public ResultVO<OrderVO> getOrderById(@PathVariable(value = "id") Integer id)</code>	输入合法, 登录	获取相应id的订单
OrderController.pay	<code>public ResultVO<Boolean> pay(@PathVariable(value = "id") Integer id)</code>	输入合法, 用户登录	更新数据库中订单状态, 支付完成
OrderController.getOrder	<code>public ResultVO<List<OrderVO>> getOrder()</code>	登录	获取与用户身份相应的订单

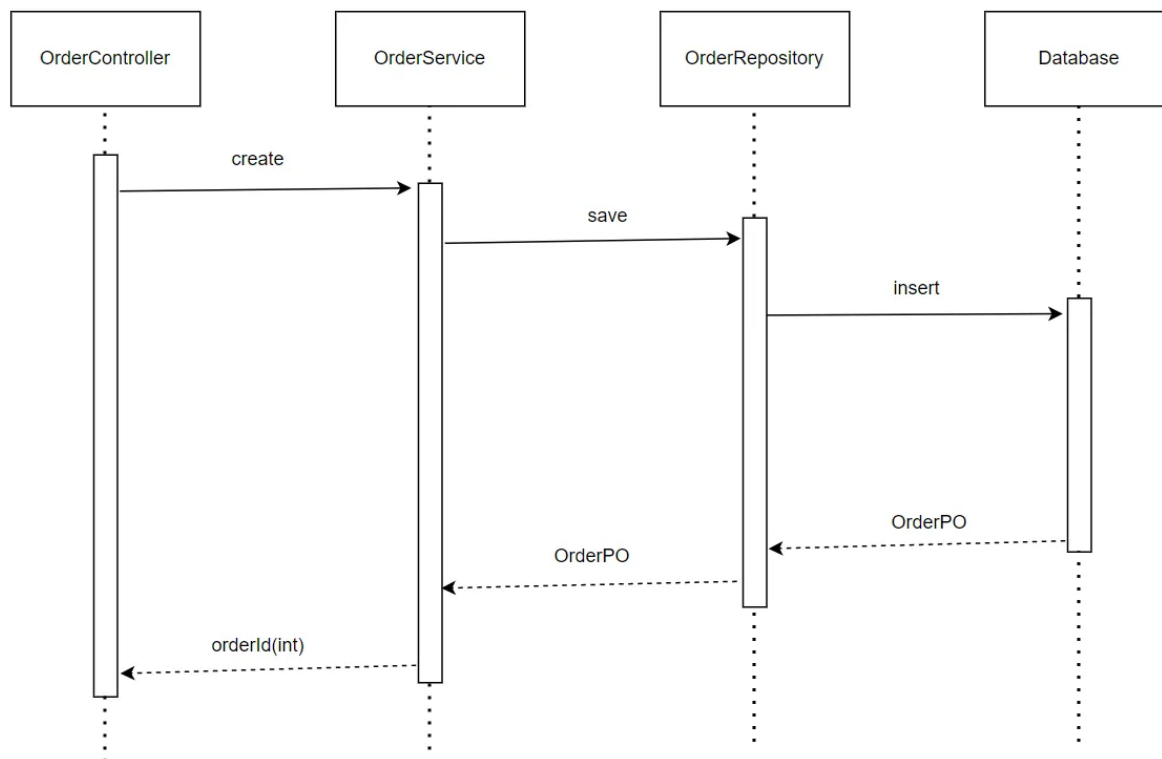
OrderController.delivery	public ResultVO<Boolean> delivery(@PathVariable(value = "id") Integer id)	输入合法, 员工登录	更新数据库中订单状态, 发货完成
OrderController.receive	public ResultVO<Boolean> receive(@PathVariable(value = "id") Integer id)	输入合法, 顾客登录	更新数据库中订单状态, 收货完成
OrderController.comment	public ResultVO<Boolean> comment(@PathVariable(value = "id") Integer id, @RequestBody CommentVO comment)	输入合法, 顾客登陆	将添加的评论传给数据库

服务接口:

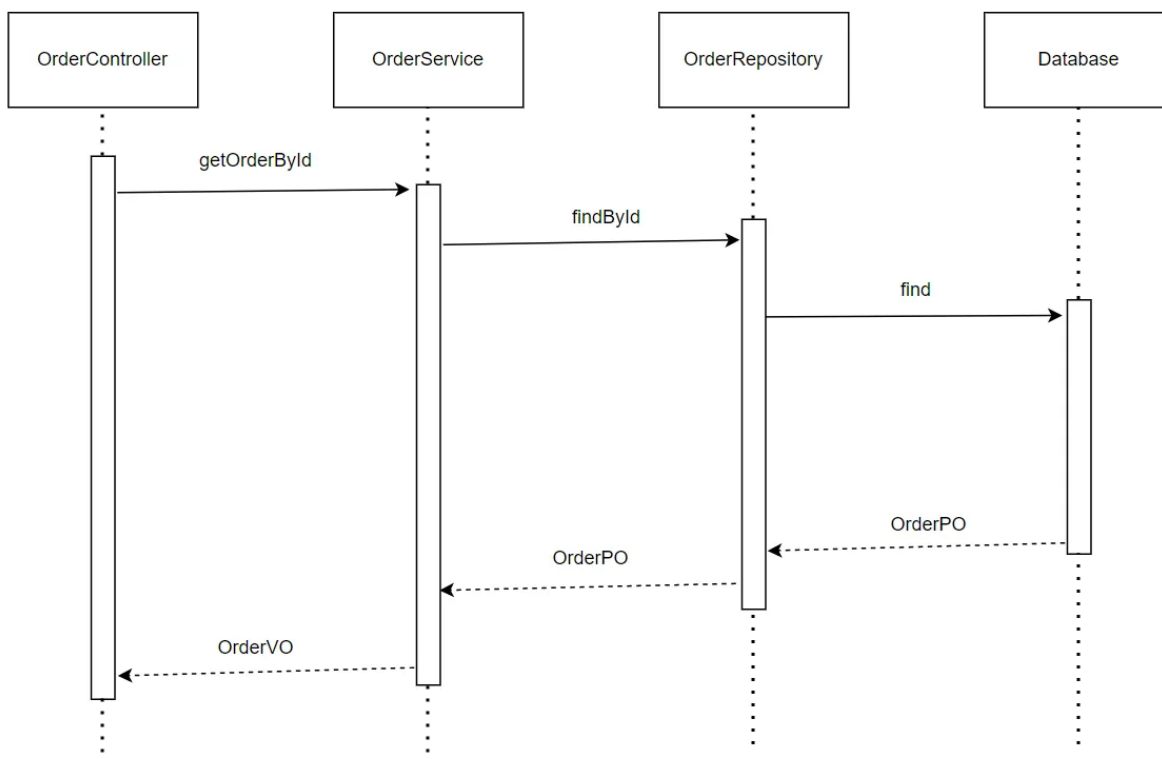
语法	服务
OrderController.create(OrderVO order)	将添加的订单传给数据库
OrderController.getOrderById(Integer id)	获取相应id的订单
OrderController.pay(Integer id)	更新数据库中订单状态, 支付完成
OrderController.getOrder()	获取与用户身份相应的订单
OrderController.delivery(Integer id)	更新数据库中订单状态, 发货完成
OrderController.receive(Integer id)	更新数据库中订单状态, 收货完成
OrderController.comment(Integer id, CommentVO comment)	将添加的评论传给数据库

4.2.4.4 业务逻辑层的动态模型

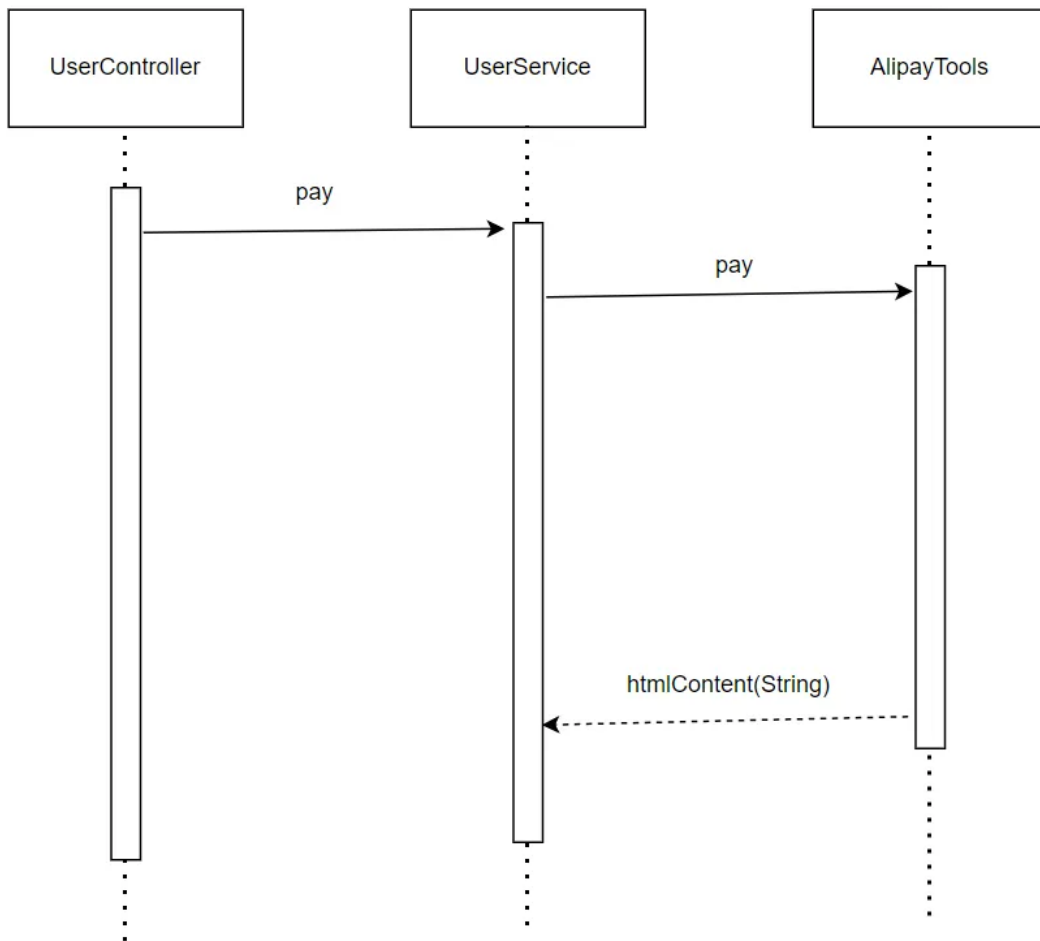
下图表明了 在 BlueWhale 系统中, OrderController.create 的相关对象之间的协作。



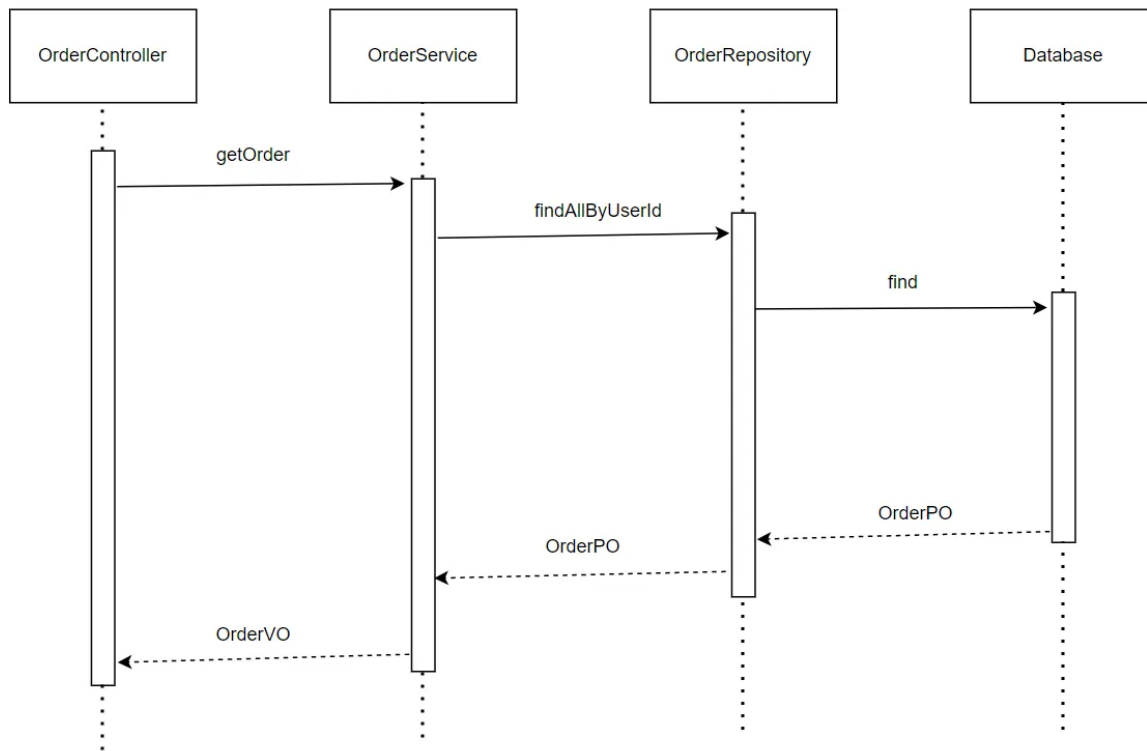
下图表明了 在 BlueWhale 系统中，OrderController.getOrderById 的相关对象之间的协作。



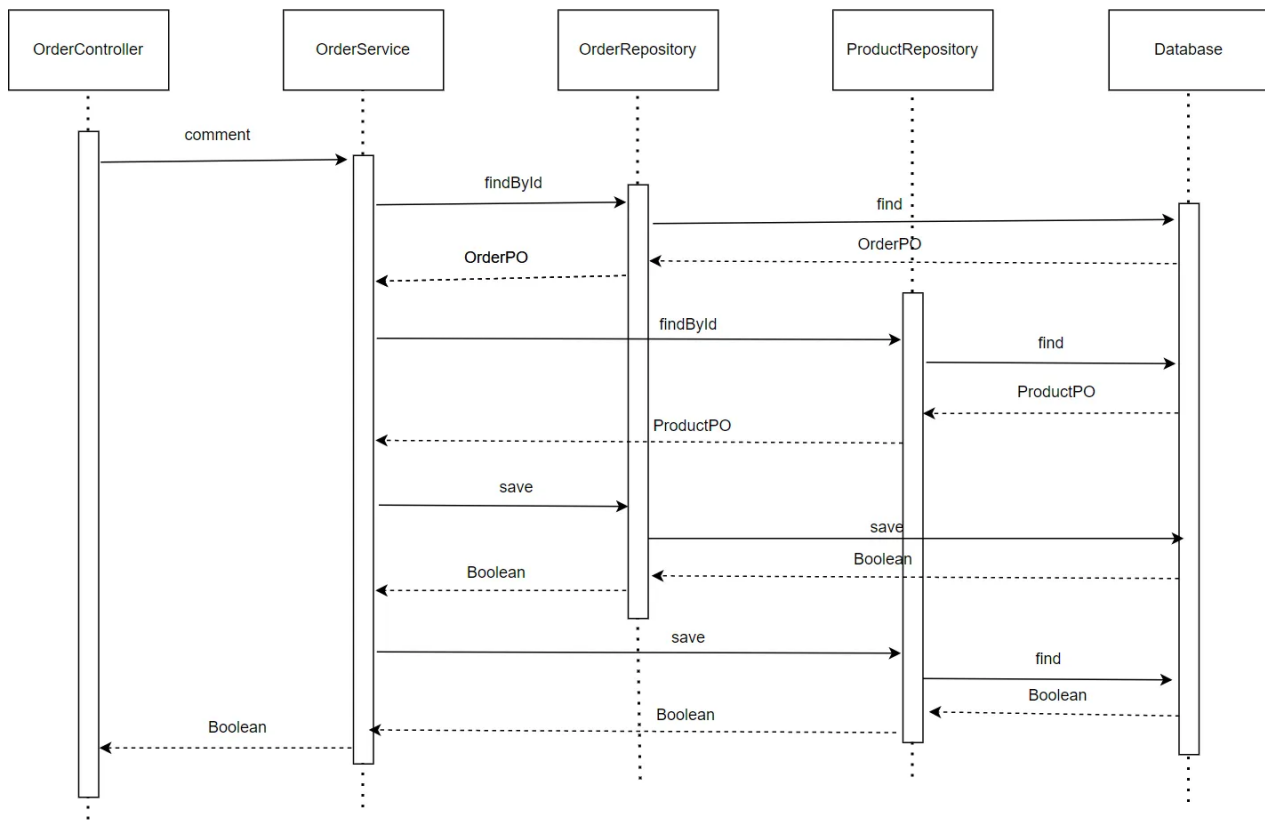
下图表明了 在 BlueWhale 系统中，OrderController.pay 的相关对象之间的协作。



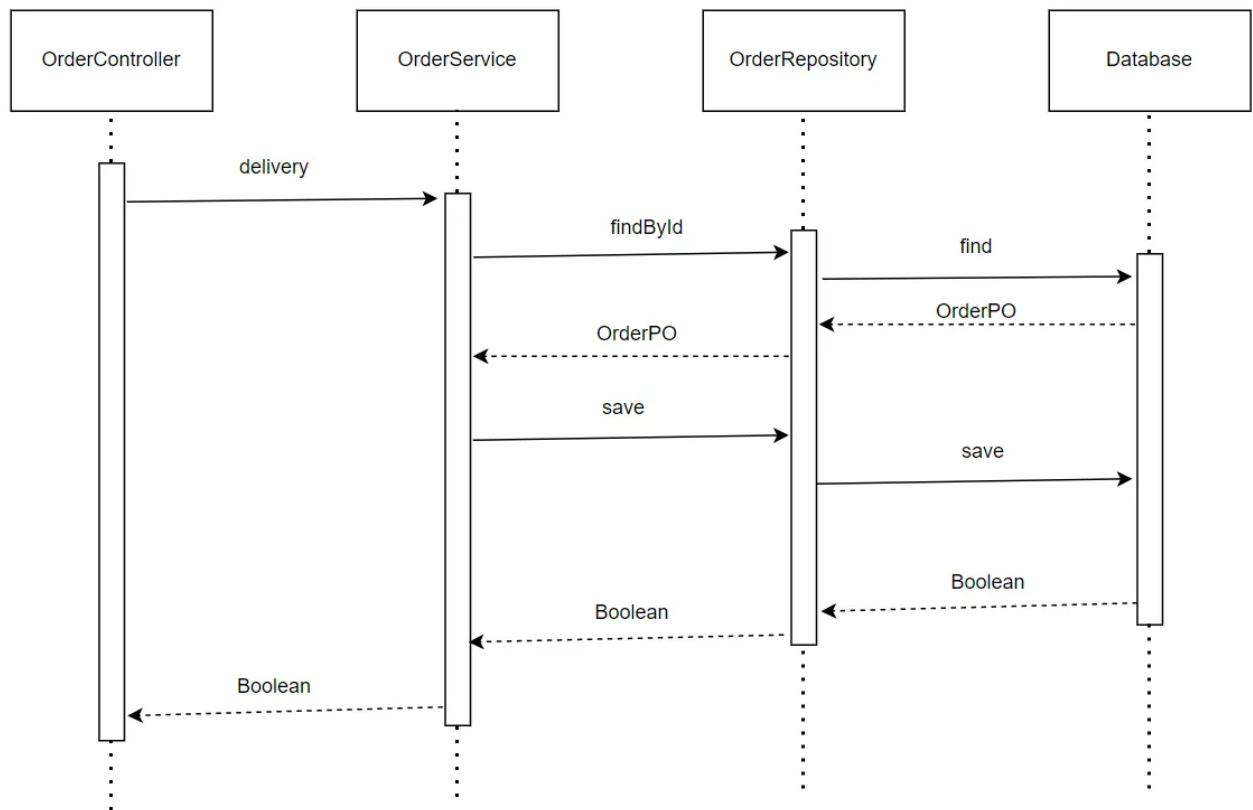
下图表明了 在 BlueWhale 系统中，OrderController.getOrder 的相关对象之间的协作。



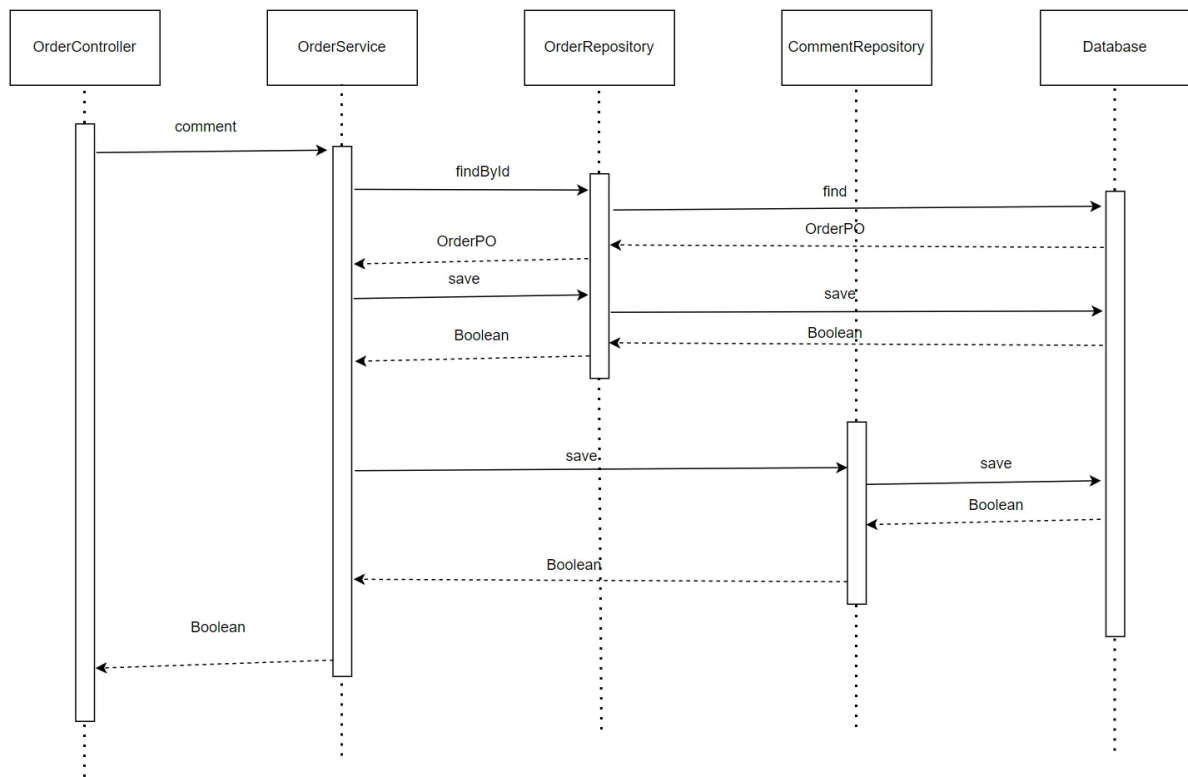
下图表明了 在 BlueWhale 系统中，OrderController.delivery 的相关对象之间的协作。



下图表明了 在 ERP 系统中，OrderController.receive 的相关对象之间的协作。



下图表明了 在 BlueWhale 系统中，OrderController.comment 的相关对象之间的协作。



4.2.4.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.2.5 toolsbl

4.2.5.1 模块概述

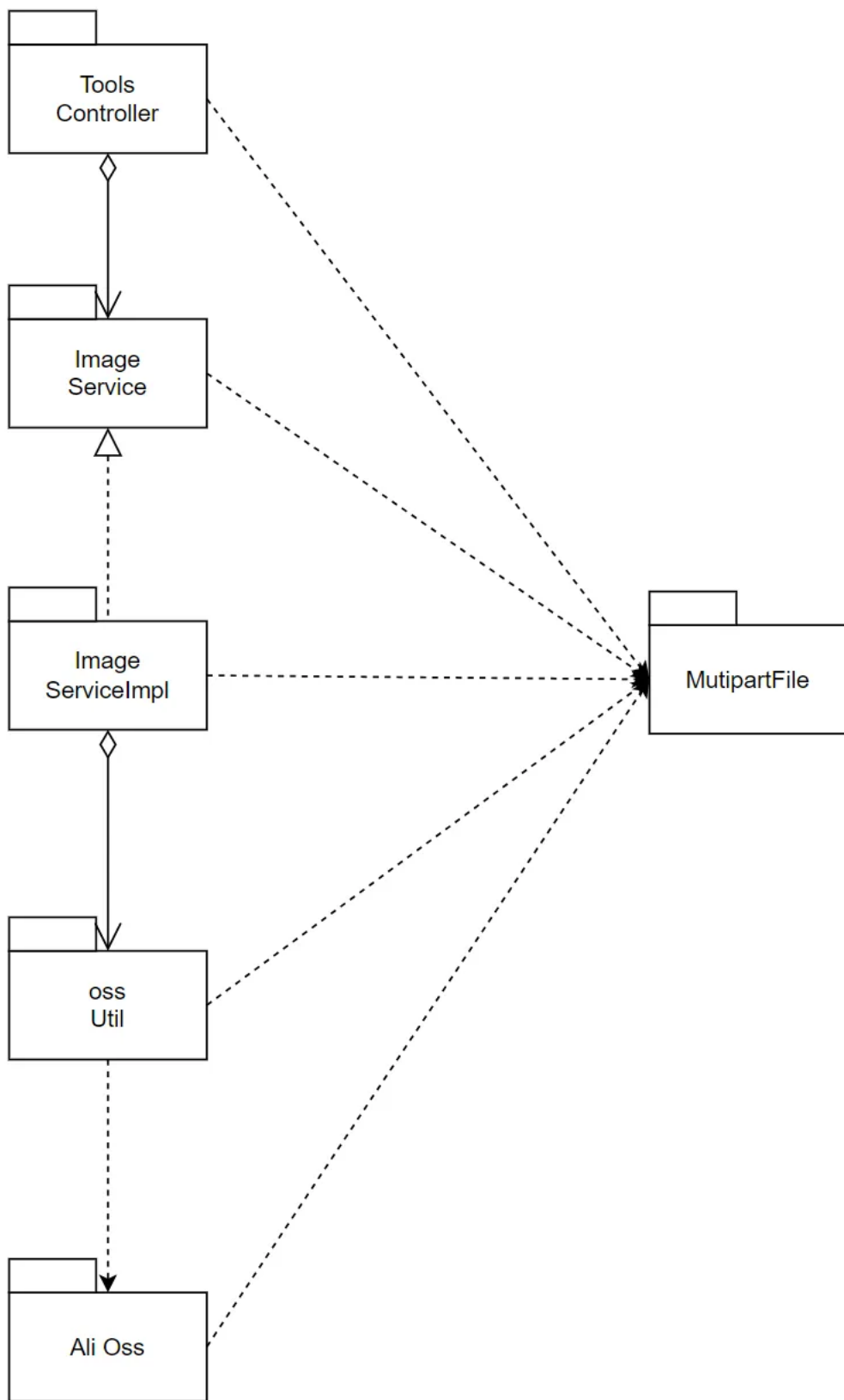
toolsbl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

toolsbl 模块的职责及接口参见软件系统结构描述文档。

4.2.5.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口

userbl的模块设计如下：



各个类的职责如下:

ToolsController	负责实现杂项信息前后端的数据交互
-----------------	------------------

ImageService	定义图片相关的接口
ImageServiceImpl	具体实现图片相关功能
OssUtil	实现对于数据库的增删改查
MultipartFile	图片文件存储载体

4.2.5.3 模块内部类的接口规范

接口规范:

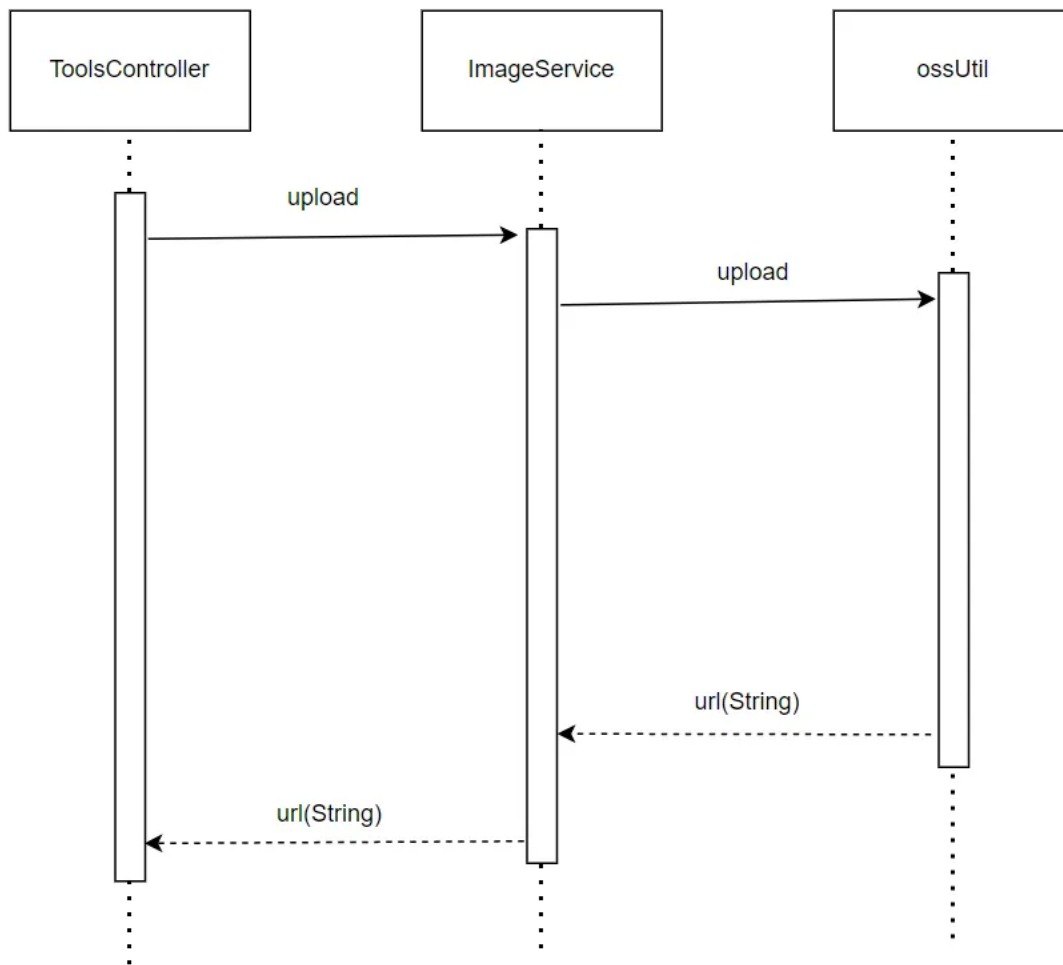
接口名称	语法	前置条件	后置条件
ToolsController.upload	public ResultVO<String> upload(@RequestParam MultipartFile file)	输入合法, 登录	将上传的照片传递给阿里oss

服务接口:

服务名	服务
ToolsController.upload(MultipartFile file)	将上传的照片传递给阿里oss

4.2.5.4 业务逻辑层的动态模型

下图表明了 在 BlueWhale 系统中, ToolsController.upload的相关对象之间的协作。



4.2.5.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.2.6 couponbl

4.2.6.1 模块概述

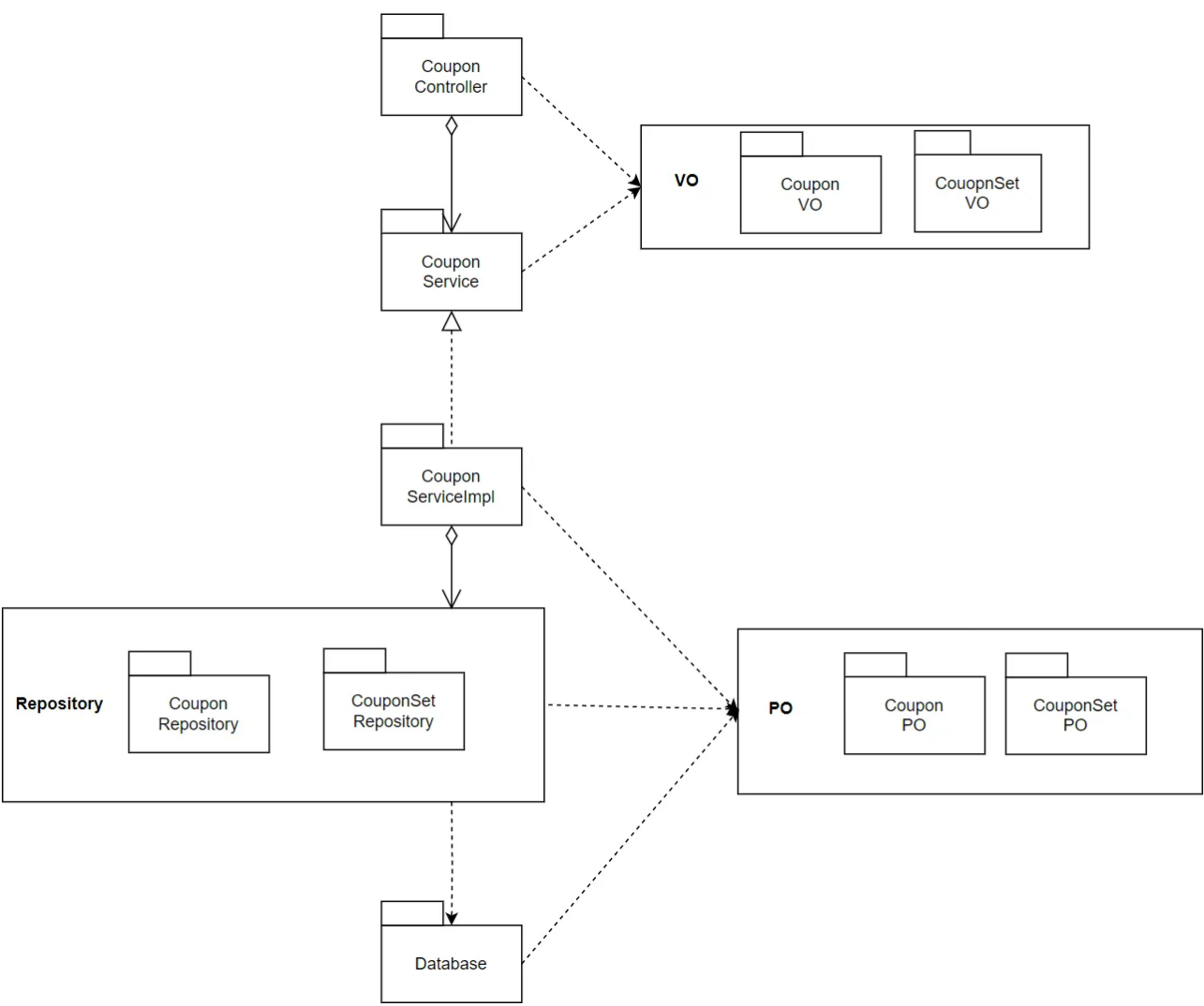
couponbl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

couponbl 模块的职责及接口参见软件系统结构描述文档。

4.2.6.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口。CouponPO 和 CouponSetPO 是作为账户信息的持久化对象而添加到设计模型中的。

couponbl的模块设计如下:



各个类的职责如下:

CouponController	负责实现优惠券信息前后端的数据交互
CouponService	定义优惠券相关的接口
CouponServiceImpl	具体实现优惠券相关功能
CouponPO	优惠券相关信息的实现载体
CouponRepository	实现对于数据库的增删改查

CouponVO	优惠券信息持久化
CouponSetPO	优惠券组相关信息的实现载体
CouponSetReportory	实现对于数据库的增删改查
CouponSetVO	优惠券组信息持久化

4.2.6.3 模块内部类的接口规范

接口规范:

接口名称	语法	前置条件	后置条件
CouponController.create	public ResultVO<Boolean> create(@RequestBody CouponSetVO couponSet)	登录为 员工或 者经理, 输入合 法	创建优惠券组
CouponController.getAllCouponSet	public ResultVO<List<Integer>> getAllCouponSet()	输入合 法	获取所有优惠券组
CouponController.getAllCoupon	public ResultVO<List<CouponVO>> getAllCoupon(@PathVariable(value = "getType")GetCouponEnum getType)	顾客登 录, 输入 合法	获取顾客优惠券列表
CouponController.check	public ResultVO<Boolean> check(@PathVariable(value = "setId") Integer setId)	输入合 法, 登录 为顾客	检查优惠券组是否已经 被顾客领取
CouponController.getById	public ResultVO<CouponSetVO> getById(@PathVariable(value = "setId") Integer setId)	输入合 法, 登录	获取优惠券组的信息

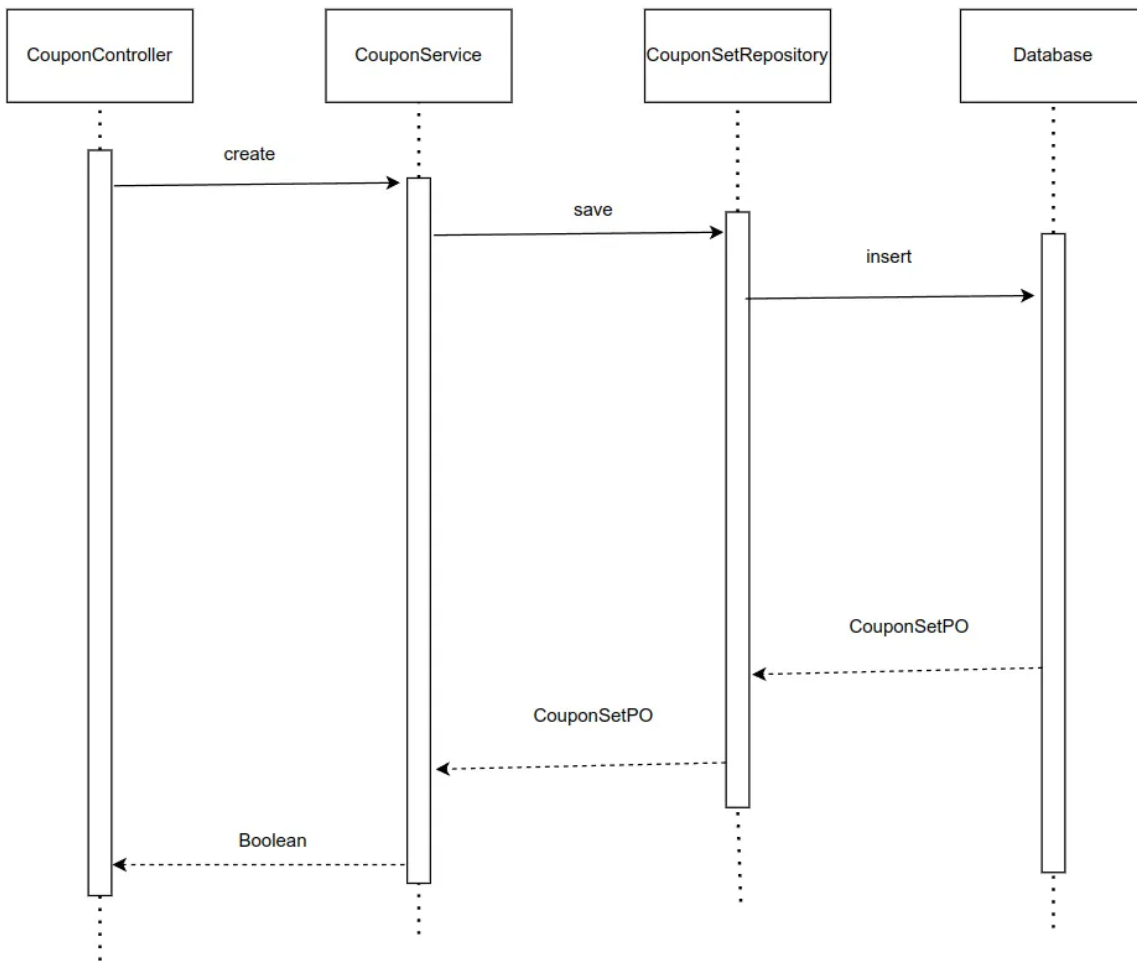
CouponController.acquire	public ResultVO<CouponVO> acquire(@PathVariable(value = "setId") Integer setId)	顾客登录, 输入合法	顾客领取优惠券
CouponController.isValid	public ResultVO<Boolean> isValid(@PathVariable(value = "setId") Integer setId)	输入合法	检查优惠券组是否过期

服务接口:

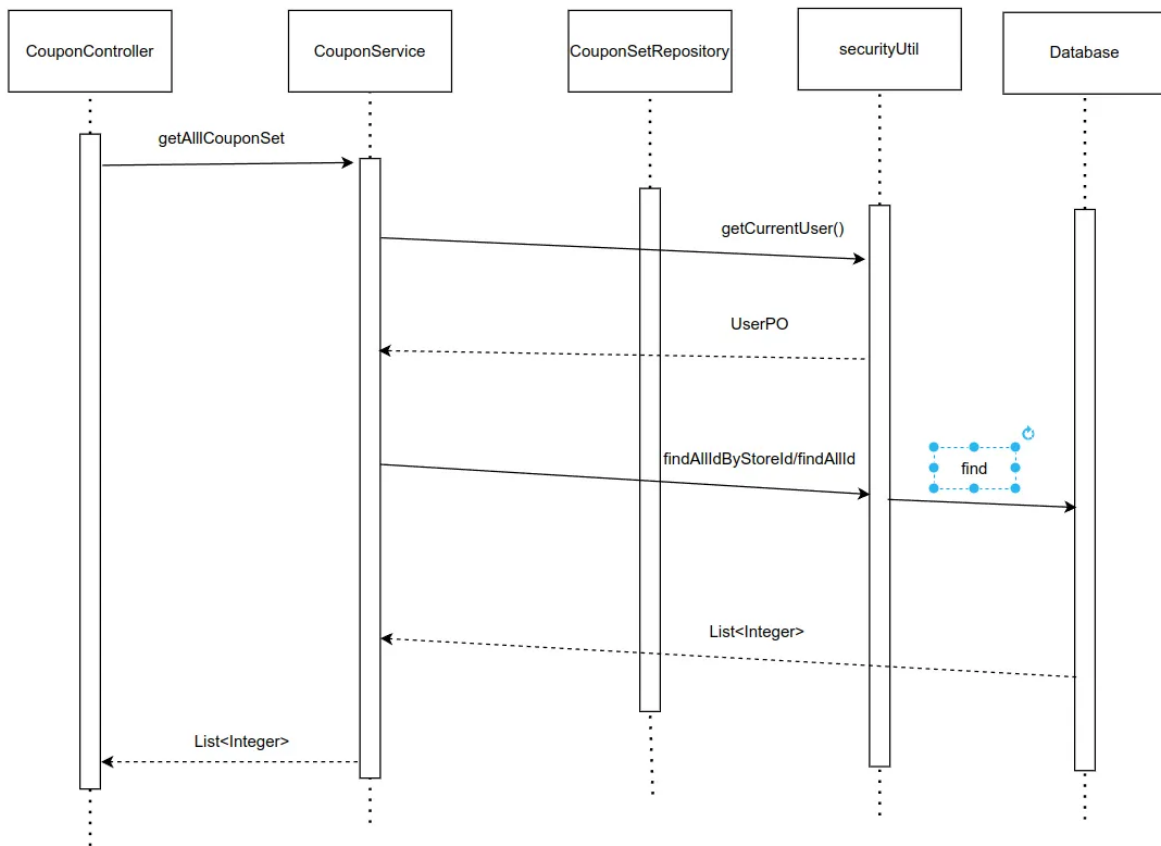
服务名	服务
CouponController.create(CouponSetVO couponSet)	创建优惠券组
CouponController.getAllCouponSet()	获取所有优惠券组
CouponController.getAllCoupon(GetCouponEnum getType)	获取顾客优惠券列表
CouponController.check(Integer setId)	检查优惠券组是否已经被顾客领取
CouponController.get(@PathVariable(Integer setId)	获取优惠券组的信息
CouponController.acquire(Integer setId)	顾客领取优惠券
CouponController.isValid(Integer setId)	检查优惠券组是否过期

4.2.6.4 业务逻辑层的动态模型

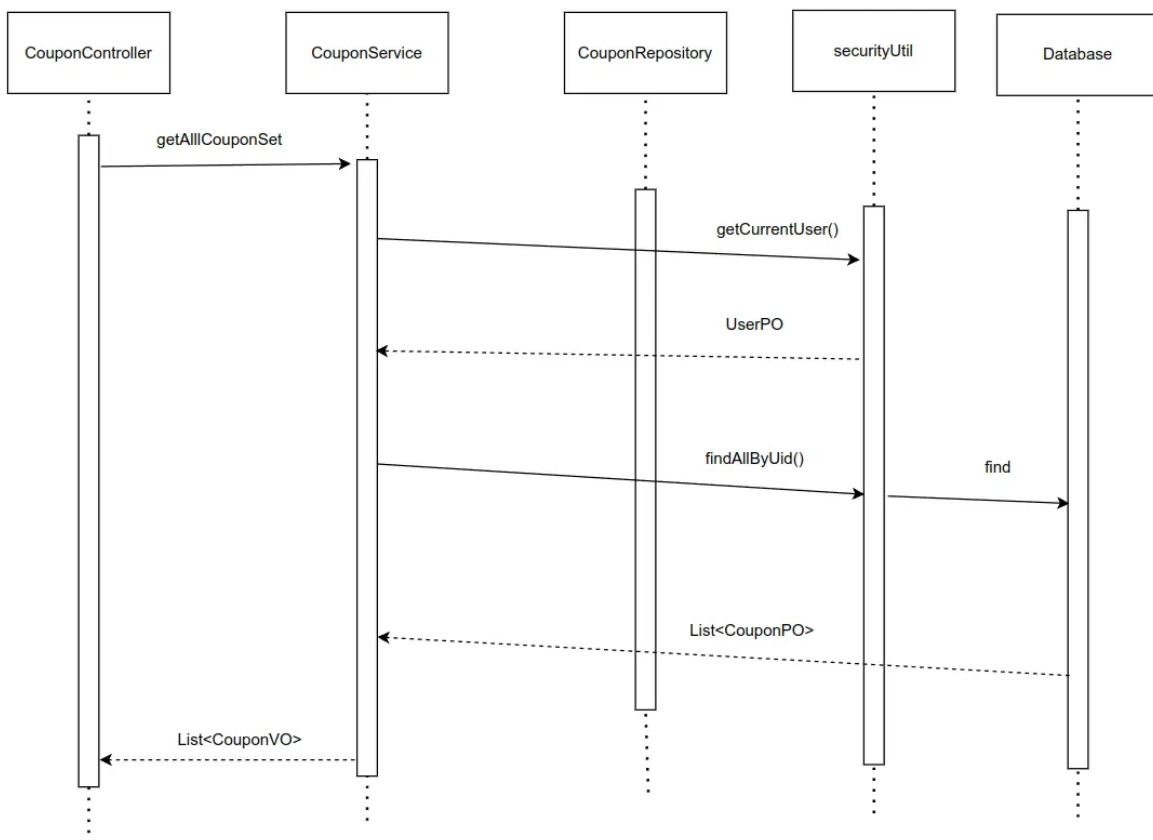
下图表明了 在 BlueWhale 系统中, CouponController.create 的相关对象之间的协作。



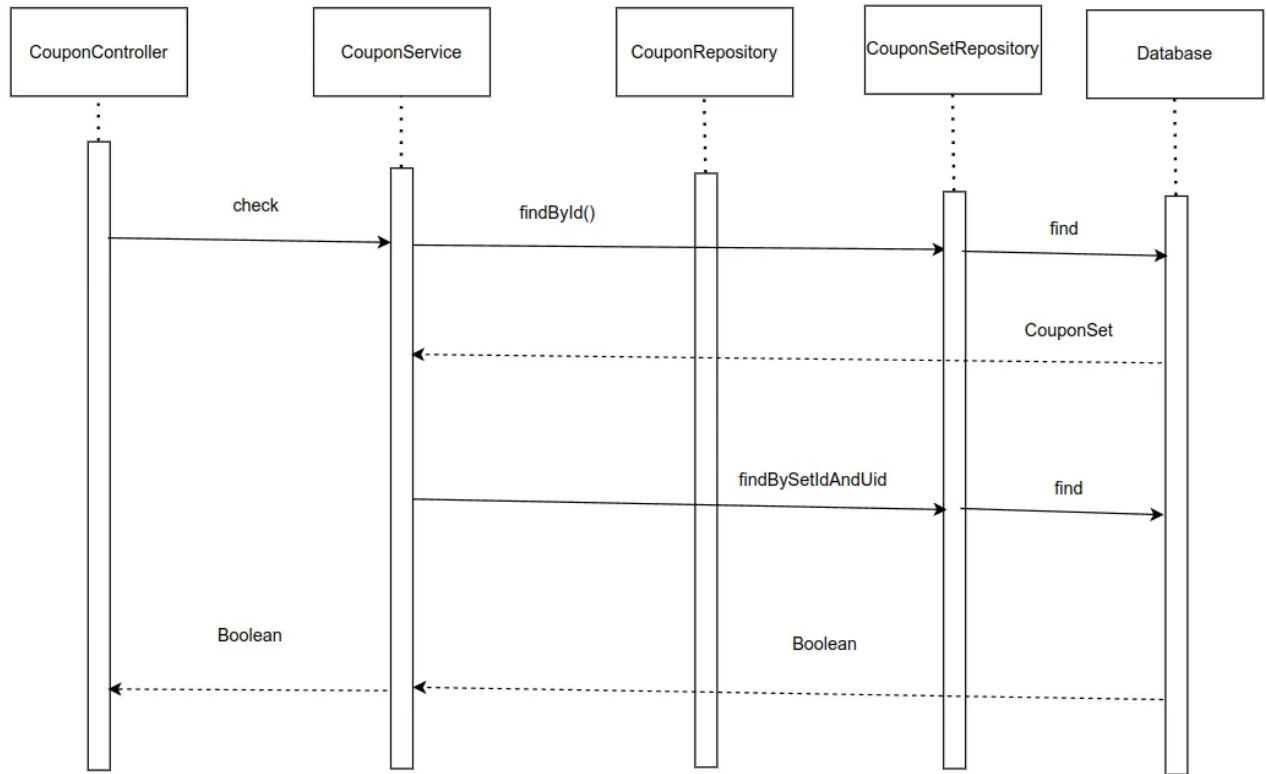
下图表明了 在 BlueWhale 系统中，`CouponController.getAllCouponSet` 的相关对象之间的协作。



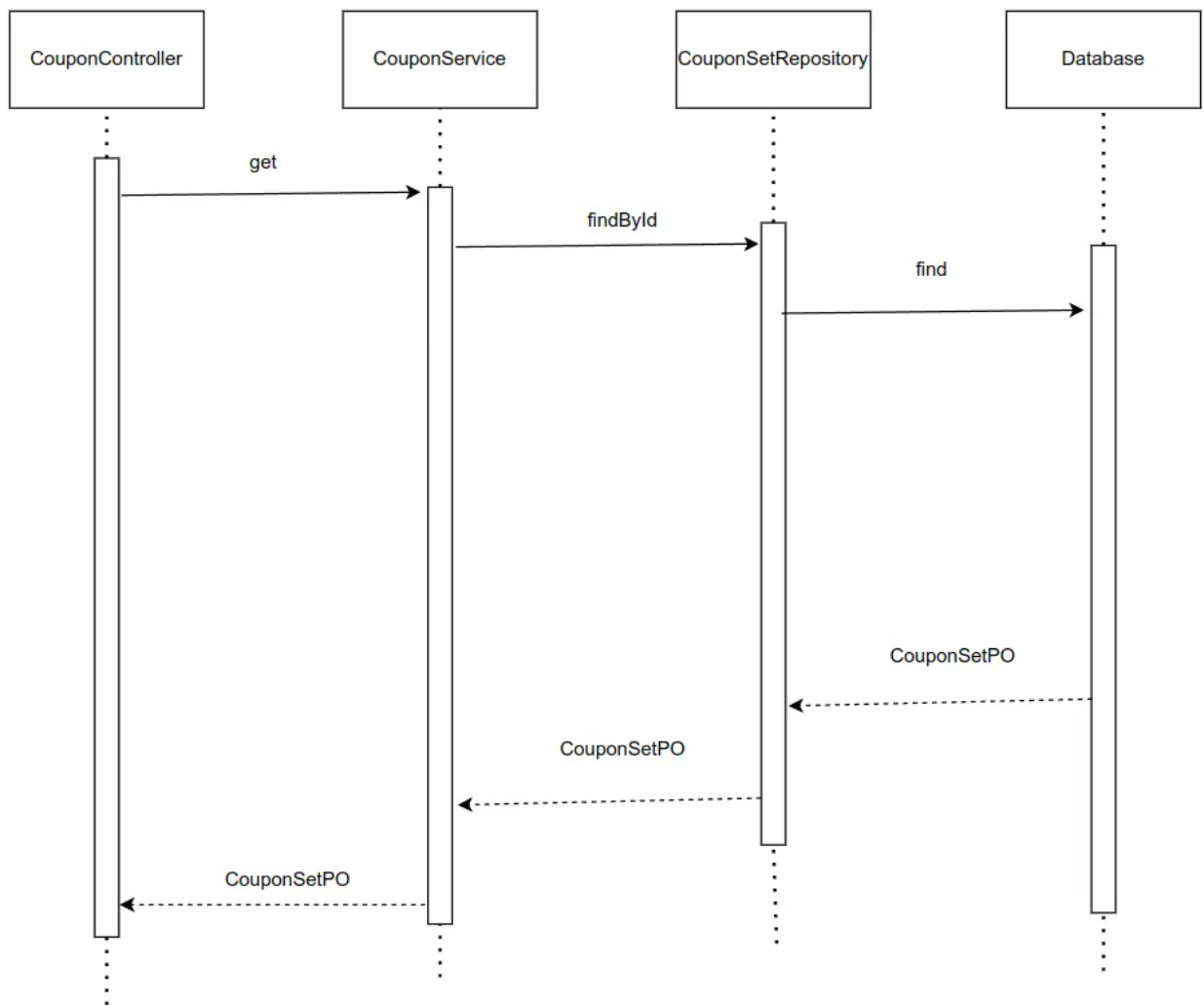
下图表明了 在 BlueWhale 系统中，CouponController.getAllCoupon的相关对象之间的协作。



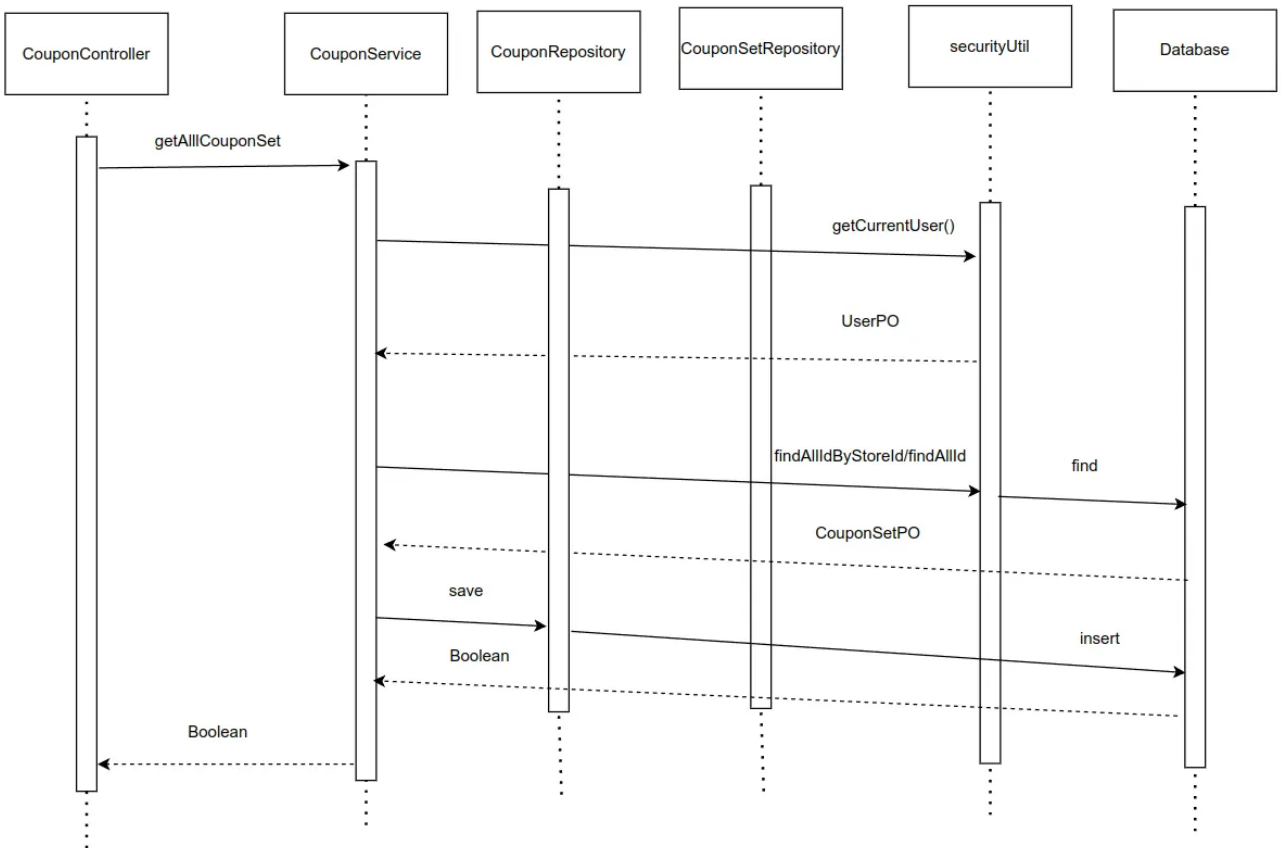
下图表明了 在 BlueWhale 系统中，CouponController.check 的相关对象之间的协作。



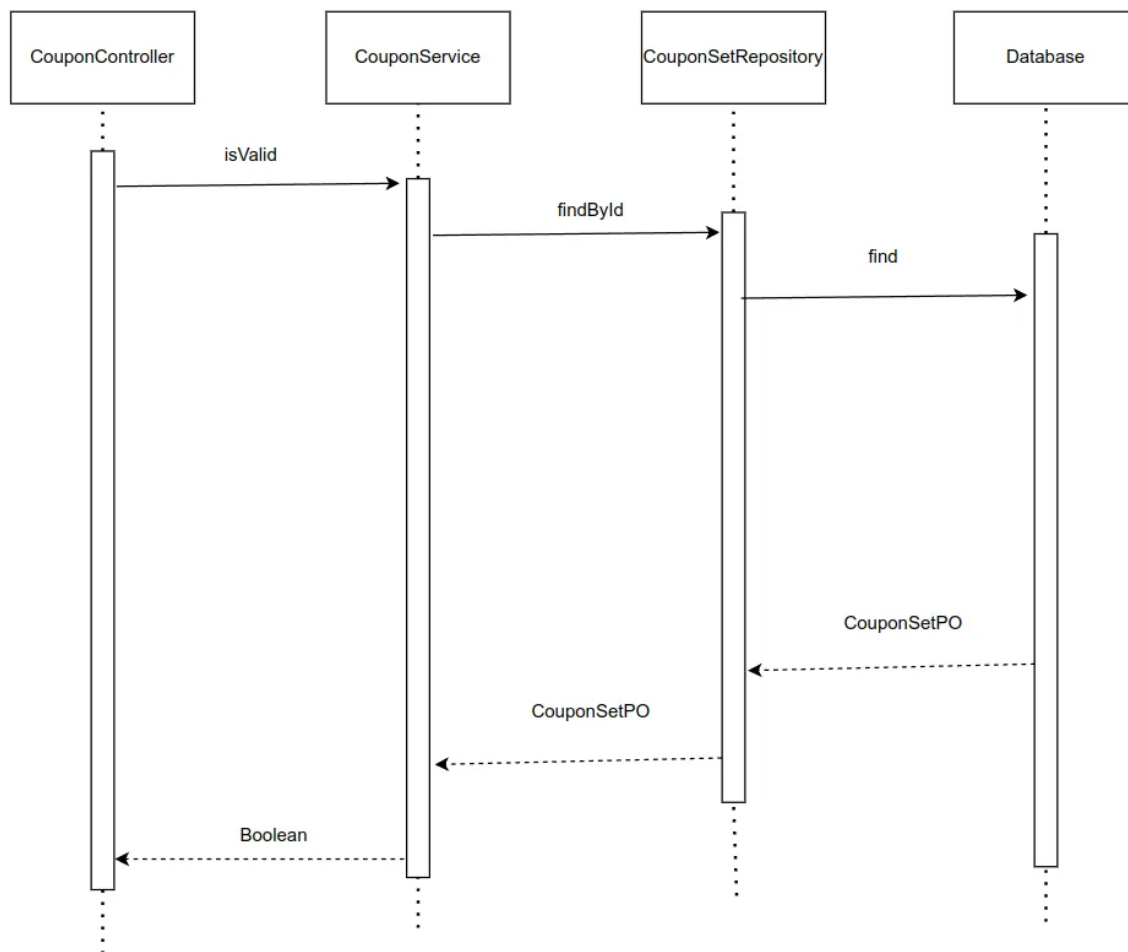
下图表明了 在 BlueWhale 系统中，CouponController.get 的相关对象之间的协作。



下图表明了 在 BlueWhale 系统中，`CouponController.acquire` 的相关对象之间的协作。



下图表明了 在 BlueWhale 系统中，`CouponController.isValid` 的相关对象之间的协作。



4.2.6.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.2.7 apipaybl

4.2.7.1 模块概述

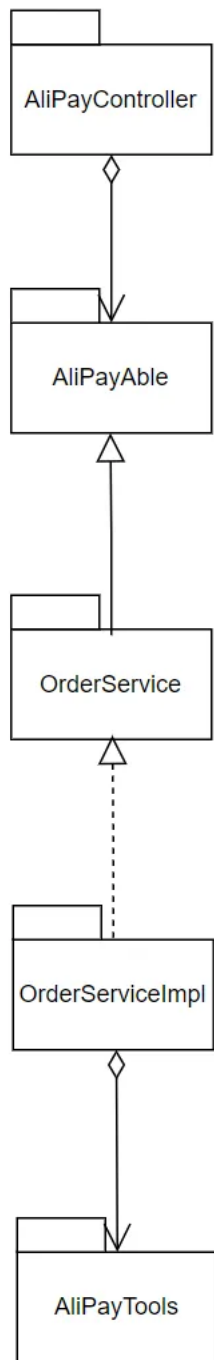
apipaybl 模块承担的需求参见需求规格说文档功能需求及相关非功能需求说明文档。

apipaybl 模块的职责及接口参见软件系统结构描述文档。

4.2.7.2 整体结构

根据体系结构的设计，我们将系统分为了前端展示层、Controller 层、业务逻辑层、数据层。每一层为了增加灵活性，我们会添加接口。比如展示层和业务逻辑层之间，我们设置了 Service 接口。业务逻辑层和数据层之间设置了Mapper 接口。

AliPaybl的模块设计如下：



各个类的职责如下:

AliPayController	负责和支付宝的交互
AliPayAble	定义实现与支付宝交互的具体行为的类的接口
OrderServiceImpl	定义订单服务的接口
OrderService	订单相关事务

AliPayTools	与支付宝进行交互的相关工具
-------------	---------------

4.2.7.3 模块内部类的接口规范

接口规范:

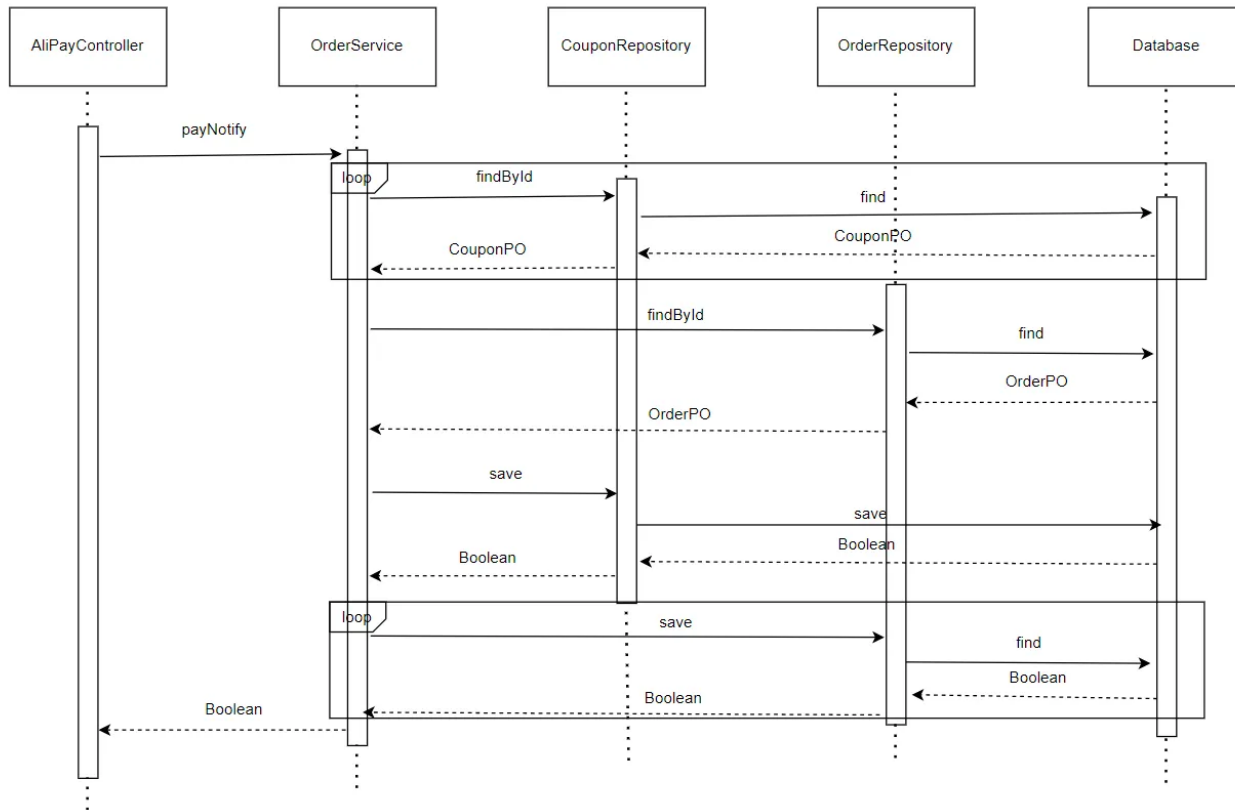
接口名称	语法	前置条件	后置条件
AliPayController.notify	public String notify(@RequestParam(value = "service") String notifyService, HttpServletRequest httpServletRequest)	输入合法	改变优惠券组, 订单状态
AliPayController.returnUrl	public void returnUrl(@RequestParam(value = "url") String url, HttpServletResponse httpServletResponse)	输入合法	导航到相应支付返回页面

服务接口:

服务名	服务
AliPayController.notify(String notifyService, HttpServletRequest httpServletRequest)	支付宝调用此接口, 改变优惠券组, 订单状态
AliPayController.returnUrl(String url, HttpServletResponse httpServletResponse)	支付宝返回后, 导航到相应支付返回页面

4.2.7.4 业务逻辑层的动态模型

下图表明了 在 BlueWhale 系统中, AliPayController.notify 的相关对象之间的协作。



4.2.7.5 业务逻辑层的设计原理

利用委托式控制风格，每个界面需要访问的业务逻辑由各自的控制器委托给不同的领域对象。

4.3 数据层的分解

数据层的开发包图参见体系结构设计文档

4.3.1 userdata

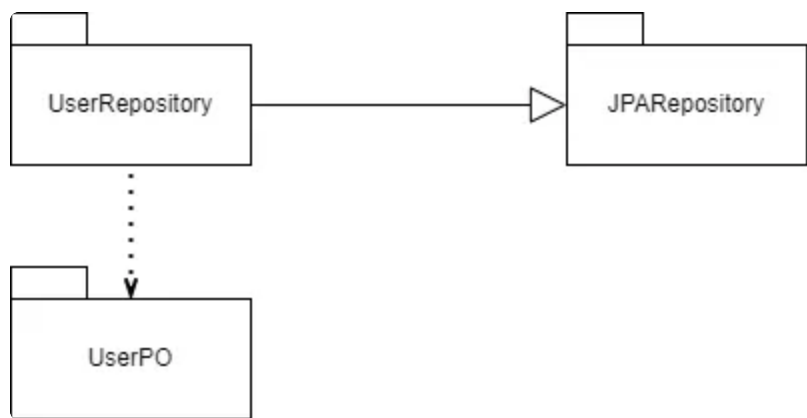
4.3.1.1 模块概述

userdata模块需求：负责用户数据的持久化处理，提供用户数据的增删改查操作。

userdata模块职责：与业务逻辑层进行交互，将用户数据存储到数据库中，并提供用户数据的读取功能。

4.3.1.2 整体结构

userdata的模块设计如下图



各个类的职责如下表

模块	职责
UserRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
UserPO	持久化对象类，对应数据库表结构。
JPARepository	提供数据库接口。

4.3.1.3 模块内部类的接口规范

userdata模块的接口规范

接口名字	语法	前置条件	后置条件
UserRepository.findByPhone	findByPhone(String phone)	账户在数据库中存在	根据手机号查询账户
UserRepository.findByPhoneAndPassword	findByPhoneAndPassword(String phone, String password)	账户在数据库中存在	根据手机号和密码查询账户

userdata模块的服务接口

服务名	服务
UserRepository.findByPhone	根据手机号查询账户
UserRepository.findByPhoneAndPassword	根据手机号和密码查询账户

4.3.2 storedata

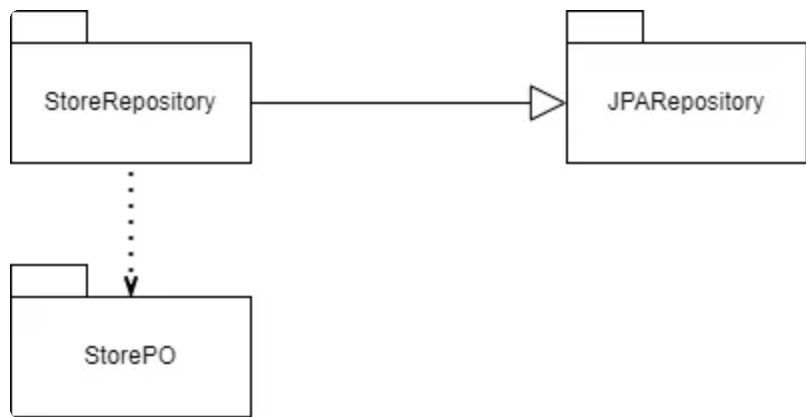
4.3.2.1 模块概述

storedata模块需求：负责商店数据的持久化处理，提供商店数据的增删改查操作。

storedata模块职责：与业务逻辑层进行交互，将商店数据存储到数据库中，并提供商店数据的读取功能。

4.3.2.2 整体结构

userdata的模块设计如下图



各个类的职责如下表

模块	职责
StoreRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
StorePO	持久化对象类，对应数据库表结构。
JPARepository	提供数据库接口。

4.3.2.3 模块内部类的接口规范

storedata模块的接口规范

接口名字	语法	前置条件	后置条件
StoreRepository.findByName(String name)	findByName(String name)	商店在数据库中存在	根据商店名查询商店

storedata模块的服务接口

服务名	服务
StoreRepository.findByName	根据商店名查询商店

4.3.3 productdata

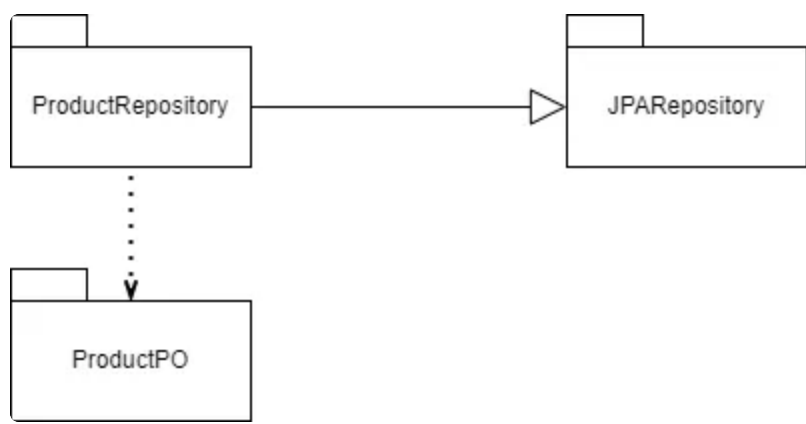
4.3.3.1 模块概述

productdata模块需求：负责商品数据的持久化处理，提供商品数据的增删改查操作。

productdata模块职责：与业务逻辑层进行交互，将商品数据存储到数据库中，并提供商品数据的读取功能。

4.3.3.2 整体结构

productdata的模块设计如下图



各个类的职责如下表

模块	职责
ProductRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
ProductPO	持久化对象类，对应数据库表结构。
JPARepository	提供数据库接口。

4.3.3.3 模块内部类的接口规范

productdata模块的接口规范

接口名字	语法	前置条件	后置条件
------	----	------	------

ProductRepository.findAllByStoreId	findAllByStoreId(Integer storeId)	商店在数据库中存在	根据商店id查询商品，返回商店所有商品
ProductRepository.findByStoreIdAndName	findByStoreIdAndName(Integer storeId, String name)	商店在数据库中存在，且有对应名字的商品	根据商店id查询给定名字的商品

productdata模块的服务接口

服务名	服务
ProductRepository.findAllByStoreId	根据商店id查询商品，返回商店所有商品
ProductRepository.findByStoreIdAndName	根据商店id查询给定名字的商品

4.3.4 orderdata

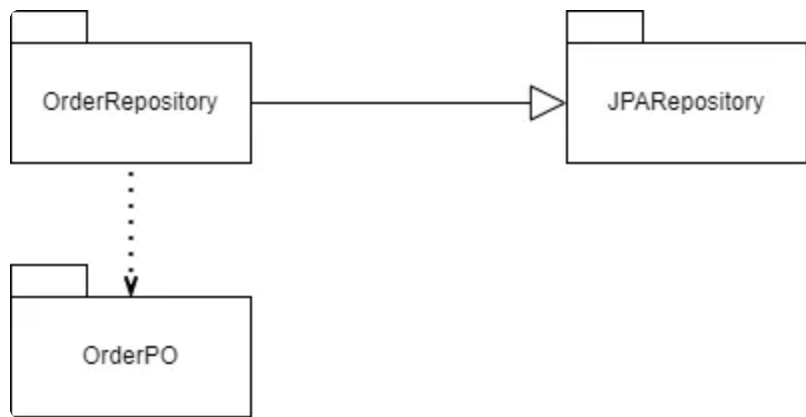
4.3.4.1 模块概述

orderdata模块需求：负责订单数据的持久化处理，提供订单数据的增删改查操作。

orderdata模块职责：与业务逻辑层进行交互，将订单数据存储到数据库中，并提供订单数据的读取功能。。l/l'，方法v擦干净。，

4.3.4.2 整体结构

orderdata的模块设计如下图



各个类的职责如下表

模块	职责
----	----

OrderRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
OrderPO	持久化对象类，对应数据库表结构。
JpaRepository	提供数据库接口。

4.3.4.3 模块内部类的接口规范

orderdata模块的接口规范

接口名字	语法	前置条件	后置条件
OrderRepository.findAllByUserId	findAllByUserId(Integer userId)	用户在数据库中存在	根据用户id查询用户发起过的订单
OrderRepository.findAllByStoreId	findAllByStoreId(Integer storeId)	商店在数据库中存在	根据商店id查找与商店有关的订单
OrderRepository.findAllByProductId	findAllByProductId(Integer id)	商品在数据库中存在	根据商品id与商品的有关的订单

orderdata模块的服务接口

服务名	服务
OrderRepository.findAllByUserId	根据用户id查询用户发起过的订单
OrderRepository.findAllByStoreId	根据商店id查找与商店有关的订单
OrderRepository.findAllByProductId	根据商品id与商品的有关的订单

4.3.5 commentdata

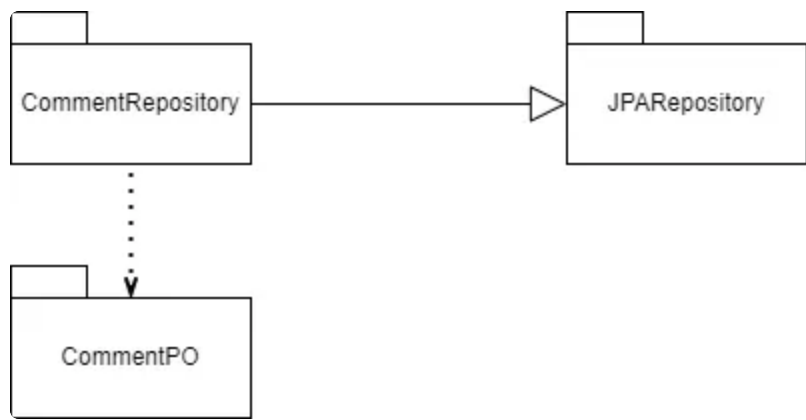
4.3.5.1 模块概述

commentdata模块需求：负责评论数据的持久化处理，提供评论数据的增删改查操作。

commentdata模块职责：与业务逻辑层进行交互，将评论数据存储到数据库中，并提供评论数据的读取功能。

4.3.5.2 整体结构

commentdata的模块设计如下图



各个类的职责如下表

模块	职责
CommentRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
CommentPO	持久化对象类，对应数据库表结构。
JPARepository	提供数据库接口。

4.3.5.3 模块内部类的接口规范

commentdata模块的接口规范

接口名字	语法	前置条件	后置条件
CommentRepository.findAllByProductId	findAllByProductId(Integer productId)	商品在数据库中存在	根据商品id查找与商品有关的评论
CommentRepository.findAllByStoreId	findAllByStoreId(Integer storeId)	商店在数据库中存在	根据商店id查找与商店有关的评论

commentdata模块的服务接口

服务名	服务
CommentRepository.findAllByProductId	根据商品id查找与商品有关的评论
CommentRepository.findAllByStoreId	根据商店id查找与商店有关的评论

4.3.6 coupondata

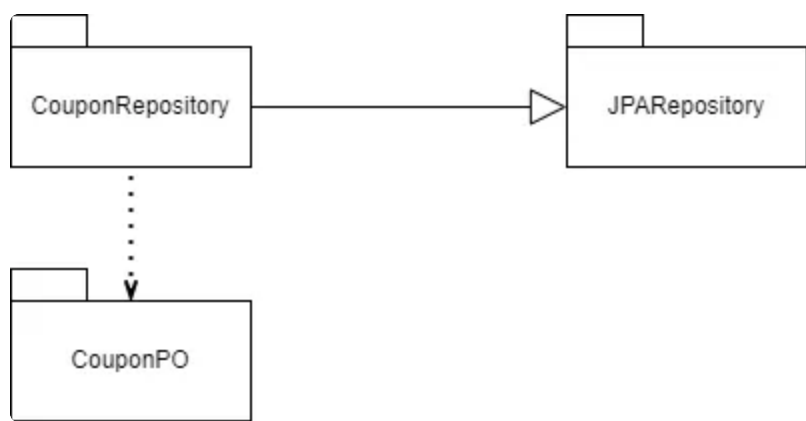
4.3.6.1 模块概述

coupondata模块需求：负责优惠券数据的持久化处理，提供优惠券数据的增删改查操作。

coupondata模块职责：与业务逻辑层进行交互，将优惠券数据存储到数据库中，并提供优惠券数据的读取功能。

4.3.6.2 整体结构

coupondata的模块设计如下图



各个类的职责如下表

模块	职责
CouponRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
CouponPO	持久化对象类，对应数据库表结构。
JPARepository	提供数据库接口。

4.3.6.3 模块内部类的接口规范

coupondata模块的接口规范

接口名字	语法	前置条件	后置条件
CouponRepository.findBySetIdAndUid	findBySetIdAndUid(Integer setId, Integer uid)	优惠券在数据库中存在	根据券组id和账户id查找某张优惠券

CouponRepository.findAllByUid	findAllByUid(Integer uid)	优惠券在数据库中存在	根据账户id查找该账户所有优惠券
CouponRepository.findAllByUidAndSetId	findAllByUidAndSetId(Integer uid, Integer setId)	优惠券在数据库中存在	根据券组id和账户id查找所有优惠券

coupondata模块的服务接口

服务名	服务
CouponRepository.findBySetIdAndUid	根据券组id和账户id查找某张优惠券
CouponRepository.findAllByUid	根据账户id查找该账户所有优惠券
CouponRepository.findAllByUidAndSetId	根据券组id和账户id查找所有优惠券

4.3.7 couponsetdata

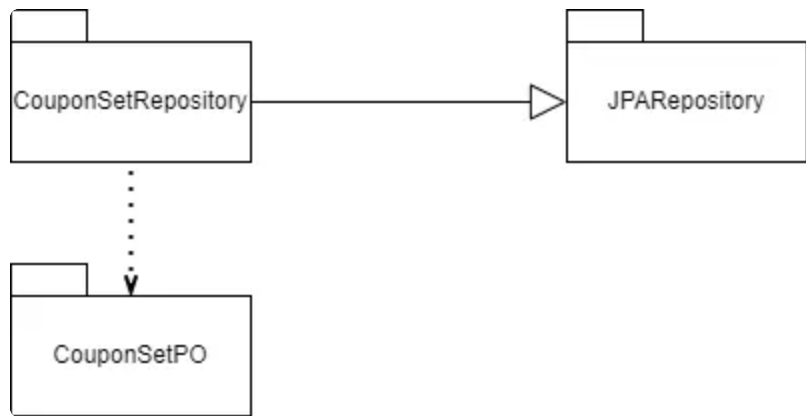
4.3.7.1 模块概述

couponsetdata模块需求：负责优惠券组数据的持久化处理，提供优惠券组数据的增删改查操作。

couponsetdata模块职责：与业务逻辑层进行交互，将优惠券组数据存储到数据库中，并提供优惠券组数据的读取功能。

4.3.7.2 整体结构

couponsetdata的模块设计如下图



各个类的职责如下表

模块	职责
----	----

CouponSetRepository	持久化数据库的接口。提供账户信息的集体载入、集体保存、增、删、改、查服务。
CouponSetPO	持久化对象类，对应数据库表结构。
JpaRepository	提供数据库接口。

4.3.7.3 模块内部类的接口规范

couponsetdata模块的接口规范

接口名字	语法	前置条件	后置条件
CouponSetRepository .findAllId	findAllId()	优惠券组在数据库中 存在	查询全局优惠券组
CouponSetRepository .findAllIdByStoreId	findAllIdByStoreId(Integer storeId)	优惠券组在数据库中 存在	根据商店id查询商店优 惠券组

couponsetdata模块的服务接口

服务名	服务
CouponSetRepository.findAllId	查询全局优惠券组
CouponSetRepository.findAllIdByStoreId	根据商店id查询商店优惠券组

依赖视角

